



Universidad Politécnica de Madrid

**Escuela Técnica Superior de Ingeniería Agronómica, Alimentaria
y de Biosistemas**

Máster Universitario en Deep Learning

Trabajo Fin de Máster

MongoMind: Un asistente conversacional de lenguaje natural a MQL para MongoDB



Autor: **Lucas Silva Pérez**

Director: **Alejandro Martín**

Barcelona, Julio de 2026

Resumen

MongoMind es un asistente conversacional que traduce preguntas en lenguaje natural, en español e inglés, a consultas MQL válidas, las ejecuta de forma segura sobre la base de datos y devuelve la respuesta en lenguaje natural. Surge porque MQL, el lenguaje de consulta de MongoDB, es verboso y exige conocimientos técnicos que la mayoría de analistas no tiene, lo que obliga a depender de perfiles especializados incluso para consultas sencillas. El sistema usa un modelo de lenguaje de gran tamaño ejecutado localmente con Ollama, sin fine-tuning, y se adapta a colecciones y datasets no vistos combinando few-shot prompting con inferencia dinámica de esquema. Incorpora además una capa de seguridad de solo lectura y memoria conversacional. Sobre un *benchmark* propio de sesenta y cinco consultas, un modelo local de tres mil millones de parámetros alcanza un 73,7 % de corrección funcional con una latencia media de 1,27 segundos, a la altura de modelos varias veces mayores y mucho más lentos, lo que confirma que la adaptación por *prompt* basta para resolver la tarea sin reentrenamiento.

Palabras clave: NL2MQL, text-to-query, LLM local, prompt engineering, in-context learning, MongoDB, Ollama.

Abstract

MongoMind is a conversational assistant that translates natural language questions, in Spanish and English, into valid MQL queries, executes them safely against the database, and returns the answer in natural language. It addresses the fact that MQL, MongoDB's query language, is verbose and requires technical knowledge that most analysts lack, forcing constant reliance on specialized staff even for simple queries. The system runs a large language model locally through Ollama, without fine-tuning, and adapts to unseen collections and datasets by combining few-shot prompting with dynamic schema inference. It also includes a read-only security layer and conversational memory. On a purpose-built benchmark of sixty-five queries, a local three-billion-parameter model reaches 73.7 % functional accuracy at an average latency of 1.27 seconds, matching models several times larger and far slower, which confirms that prompt-based adaptation suffices to solve the task without any retraining.

Keywords: NL2MQL, text-to-query, local LLM, prompt engineering, in-context learning, MongoDB, Ollama.

Agradecimientos

Quiero expresar mi gratitud a mi tutor, Alejandro Martín, por la orientación en la elección del trabajo. Agradezco también a la Universidad Politécnica de Madrid y al profesorado de máster la formación recibida durante este año, que ha sentado las bases sobre las que se apoya este Trabajo de Fin de Máster. Mención especial merece la comunidad del software libre. Este proyecto se sostiene sobre modelos de lenguaje de código abierto, ejecutados en local mediante Ollama, que han permitido construir un asistente completo sin depender de servicios de pago ni ceder los datos a terceros. Esa apertura ha hecho viable la experimentación de forma reproducible y sin coste, y encarna el espíritu de conocimiento compartido que hace posible un trabajo como este. Extiendo el agradecimiento a los proyectos MongoDB, PyMongo, FastAPI y al resto de bibliotecas abiertas que forman la columna vertebral del sistema. Por último, al Hospital del Mar de Barcelona, y en particular a la Unidad de Inteligencia Artificial, por el guiado y la resolución de muchas de las dudas de este proyecto.

Índice

Resumen	1
Abstract	2
Agradecimientos	3
1 Introducción	8
1.1 Motivación y contexto	8
1.2 Planteamiento del problema	8
1.3 Objetivos	9
1.4 Alcance y limitaciones	9
2 Estado del arte	10
2.1 Large Language Models	10
2.1.1 La arquitectura Transformer	10
2.1.2 Preentrenamiento autorregresivo	11
2.1.3 Escalado y capacidades emergentes	13
2.2 Interfaces de lenguaje natural a bases de datos (NLIDB)	13
2.3 Text-to-SQL: benchmarks y métricas de evaluación	14
2.3.1 Exact match accuracy	14
2.3.2 Execution accuracy	15
2.4 Del SQL al NoSQL/MQL	16
2.4.1 Documentos anidados, heterogéneos y sin esquema impuesto	16
2.4.2 Tipado heterogéneo dentro de un mismo campo	16
2.4.3 El <i>aggregation pipeline</i>	17
2.4.4 Ausencia de un benchmark canónico	17
2.5 LLM para generación de código y consultas	18
2.6 In-context learning y few-shot prompting frente a <i>fine-tuning</i>	18
3 Análisis del problema y requisitos	20
3.1 Análisis del problema	20
3.2 Requisitos funcionales	21
3.3 Requisitos no funcionales	21
3.4 Requisitos de seguridad	22
4 Conjuntos de Datos	23
5 Diseño de la solución	25
5.1 Visión general y arquitectura	25
5.2 Módulos del <i>core</i>	27
5.3 Capa web (FastAPI)	28
5.4 Seguridad y robustez	29
5.5 Memoria conversacional	29
6 Implementación	31
6.1 Stack tecnológico	31
6.2 Generación de MQL	31
6.2.1 Construcción del contexto y partición del prompt	32

6.2.2	Invocación del modelo	32
6.2.3	Extracción de la consulta.	32
6.2.4	Reparación de JSON <i>casi válido</i>	32
6.3	Inferencia dinámica de esquema	32
6.4	Soporte multi-dataset	33
6.5	Ingeniería de prompts	34
6.6	Despliegue con Docker	34
7	Resultados	36
7.1	El conjunto de evaluación	36
7.2	Rendimiento del modelo principal	36
7.3	Análisis de errores y refinamiento de los ejemplos	37
7.4	Comparativa entre modelos locales	38
8	Conclusiones	41
8.1	Limitaciones	41
8.2	Trabajo futuro	42
8.3	Conclusión final	42
9	Referencias	44
	Referencias	44
10	Anexos	47
10.1	Selección del <i>benchmark</i> de evaluación.	47
10.2	Plantilla <i>few-shot</i> completa de la colección movies.	47
10.3	Ejemplo de esquema inferido dinámicamente.	47
10.4	Trazas de interacción.	47
10.5	Comparativa numérica de modelos.	47
10.6	Interfaz web del asistente.	48

Índice de cuadros

Tabla 1	Subproblemas que componen el pipeline de MongoMind, del lenguaje natural al resultado.	20
Tabla 2	Stack tecnológico utilizado en el proyecto.	31
Tabla 3	Distribución de los sesenta y cinco pares del <i>benchmark</i> por nivel de complejidad e idioma.	49
Tabla 4	Muestra del nivel simple . Resultado esperado: documentos.	49
Tabla 5	Muestra del nivel medio . La columna <i>Tipo</i> indica el resultado esperado.	50
Tabla 6	Muestra del nivel alto , con etapas de <i>unwind</i> , <i>lookup</i> y agregación.	51
Tabla 7	Muestra del nivel ambiguo o con errata . Las preguntas conservan las erratas originales para evaluar la robustez del sistema.	52
Tabla 8	Reglas estrictas y pistas de la plantilla <i>few-shot</i> de movies	52
Tabla 9	Esquema de la colección movies . La colección relacionada comments referencia movies._id mediante movie_id (ObjectId) y aporta los campos name , email , movie_id , text y date	53
Tabla 10	Ejemplos verificados de pregunta y consulta incluidos en la plantilla <i>few-shot</i>	54
Tabla 11	Esquema inferido de la colección comments a partir de 100 documentos muestreados.	54
Tabla 12	Trazas de interacción. La traza 2 es una conversación de dos turnos (2a y 2b) que ejercita la memoria conversacional; la traza 3 muestra el fallo controlado ante un operador destructivo.	55
Tabla 13	Comparativa de modelos sobre el conjunto de prueba del <i>benchmark</i> . El modelo principal, llama3.2 , aparece resaltado y encabeza la corrección funcional con la menor latencia. En negrita, el mejor valor de cada magnitud destacada.	55

Índice de figuras

Figura 1	Arquitectura Transformer original. El modelo se compone de un codificador (<i>encoder</i> , izquierda) y un decodificador (<i>decoder</i> , derecha), ambos formados por una pila de N capas. El codificador combina mecanismos de autoatención multi-cabeza (<i>Multi-Head Self-Attention</i>) y redes <i>Feed Forward</i> , mientras que el decodificador incorpora además una capa de autoatención enmascarada (<i>Masked Multi-Head Attention</i>) para preservar la naturaleza autorregresiva de la generación. Las representaciones de entrada y salida se enriquecen mediante <i>embeddings</i> y codificación posicional (<i>Positional Encoding</i>), y la predicción final se obtiene a través de una capa lineal seguida de una función <i>Softmax</i> . [29]	11
Figura 2	Proceso de tokenización. El texto de entrada se segmenta en unidades discretas (<i>tokens</i>) mediante un algoritmo de subpalabras (p. ej., <i>Byte Pair Encoding</i>), y cada <i>token</i> se asocia a un identificador entero del vocabulario. Estos identificadores constituyen la entrada que el modelo transforma posteriormente en <i>embeddings</i> . [7]	12
Figura 3	Ejemplo de consultas semánticamente equivalentes (pseudocódigo).	15

Figura 4	La base relacional impone un esquema rígido declarado de antemano (columnas y tipos uniformes en todas las filas), mientras que la documental admite un esquema flexible, con campos heterogéneos, ausencias y estructuras anidadas dentro de una misma colección.	17
Figura 5	Grafo de relaciones del dataset <i>sample_mflix</i> , con sus cuatro componentes desconectados y los vínculos entre colecciones [36].	23
Figura 6	Arquitectura del <i>pipeline</i> de MongoMind: traducción de lenguaje natural a MQL (NL→MQL), validación de solo lectura y ejecución contra MongoDB Atlas mediante un LLM local servido con Ollama.	26
Figura 7	Detección de colección y <i>routing</i> multi-dataset en MongoMind. El registro cerrado de datasets actúa como frontera de seguridad, donde <code>detect_collection()</code> puntúa la pregunta contra las <i>keywords</i> de cada colección y selecciona la de mayor score (o reutiliza la anterior en <i>follow-ups</i>).	28
Figura 8	Sanitización de la entrada, limitación de frecuencia, validación del MQL como JSON, rechazo de operadores de escritura, conexión de solo lectura y registro de auditoría.	30
Figura 9	Construcción del prompt few-shot (<code>_build_messages</code>).	33
Figura 10	Topología de despliegue de MongoMind. El único componente contenedorizado es la aplicación FastAPI. El servidor Ollama se ejecuta en la máquina anfitriona y la base de datos reside en MongoDB Atlas en la nube (mediante <code>MONGODB_URI</code>). De ahí que no exista ninguna base de datos contenedorizada.	35
Figura 11	El grueso de las preguntas se concentra en los niveles simple y medio, con una cuota menor de consultas de alta complejidad y de casos ambiguos o con errata.	37
Figura 12	El predominio de los conjuntos de documentos convive con agrupaciones, recuentos y valores escalares, cada uno con su propia estrategia de comparación funcional.	37
Figura 13	La corrección funcional supera con holgura a la coincidencia exacta, lo que ilustra por qué esta última no es una medida fiel de la utilidad real.	37
Figura 14	El acierto se mantiene alto en las preguntas simples y medias y cae en las de alta complejidad, donde entran en juego las agregaciones más elaboradas.	37
Figura 15	Una única causa, la falta de guarda numérica, concentra buena parte de los errores, seguida de la omisión del despliegue de <i>arrays</i>	38
Figura 16	La mejora se observa tanto en desarrollo como en el conjunto de prueba oculto.	38
Figura 17	El modelo principal encabeza la tabla, empatado dentro del margen de variación con el mayor de todos pese a tener una fracción de sus parámetros.	39
Figura 18	El modelo mayor destaca en coincidencia exacta y el generalista más pequeño acumula la mayor tasa de error.	39
Figura 19	La precisión disminuye conforme crece la complejidad de la consulta (más operadores y etapas).	40
Figura 20	El modelo principal se ubica en la región más conveniente, con una corrección comparable a la de modelos mayores pero a una fracción de su tiempo de generación.	41
Figura 21	Interfaz web de chat de MongoMind. La pregunta en lenguaje natural, el MQL generado y los resultados se presentan juntos.	56

1. Introducción

1.1. Motivación y contexto

En la última década, las bases de datos NoSQL¹ se han convertido, en muchas organizaciones, en el modelo principal para gestionar datos heterogéneos. MongoDB, en particular, es hoy una de las soluciones más extendidas gracias a su modelo documental flexible, capaz de representar información semiestructurada, anidada y de esquema variable sin las rigideces del modelo tabular. Sin embargo, esta flexibilidad tiene una contrapartida: consultar los datos exige dominar MQL², un lenguaje sintácticamente denso y poco intuitivo para quien no procede de un perfil técnico.

Este requisito da lugar a lo que puede describirse como un cuello de botella analítico. El analista de negocio, el investigador o el especialista del área que necesita responder a una pregunta concreta sobre los datos rara vez conoce MQL, de modo que su consulta debe pasar por un intermediario técnico que la traduzca, la ejecute y le devuelva el resultado. Esta dependencia introduce latencia en la toma de decisiones, satura a los perfiles especializados con peticiones repetitivas y, en la práctica, limita el aprovechamiento real de la información disponible. El conocimiento del lenguaje se convierte así en una barrera de entrada que separa a quien tiene la pregunta de quien puede obtener la respuesta.

Paralelamente, el avance de los grandes modelos de lenguaje (LLM³) ha transformado el panorama del procesamiento del lenguaje natural y ha hecho viable un paradigma que durante años fue más una aspiración que una realidad: el *text-to-query*, es decir, la traducción automática de una pregunta formulada en lenguaje natural a una consulta ejecutable sobre una base de datos [11, 37]. Lo que tradicionalmente requería sistemas basados en reglas, gramáticas o modelos de parsing costosos de construir y mantener, hoy puede abordarse aprovechando la capacidad de generalización lingüística de los LLM. Este trabajo se sitúa precisamente en esa intersección: aplicar el paradigma *text-to-query* al ecosistema NoSQL de MongoDB para eliminar la barrera que MQL impone al usuario no técnico [14].

1.2. Planteamiento del problema

El problema que aborda este trabajo puede enunciarse de forma directa: traducir una pregunta formulada en lenguaje natural, en español o en inglés, a una consulta MQL que sea simultáneamente válida, ejecutable y segura, de manera que el usuario final obtenga la respuesta que busca sin necesidad de conocer el lenguaje de consulta subyacente.

Cada uno de esos tres adjetivos encierra una dificultad propia. Que la consulta sea válida implica que el texto generado debe respetar la sintaxis de MQL y constituir un documento JSON correcto, algo que no está garantizado cuando se genera mediante un modelo de lenguaje. Que sea ejecutable exige además que la consulta tenga sentido respecto al esquema real de los datos: que los campos referenciados existan, que los tipos sean coherentes y que las operaciones de agregación estén bien formadas. Y que sea segura

¹Base de datos no relacional que almacena información en formatos flexibles.

²*MongoDB Query Language*

³*Large Language Models*

obliga a garantizar que el sistema nunca ejecute operaciones destructivas o de escritura sobre la base de datos, por mucho que la pregunta del usuario, de manera intencionada o accidental, pudiera inducirlas. A esta triple exigencia se suma el requisito transversal de la transparencia: el usuario debe poder confiar en el resultado, lo que aconseja exponer la consulta MQL efectivamente ejecutada.

1.3. Objetivos

El objetivo general de este Trabajo Fin de Máster es el diseño e implementación de un asistente conversacional capaz de traducir lenguaje natural a MQL sobre MongoDB, ejecutar la consulta resultante y devolver una respuesta comprensible al usuario. Este objetivo general se concreta en los siguientes objetivos específicos:

- Construir un *pipeline* completo que cubra el ciclo, integrando la generación de la consulta, su ejecución contra la base de datos y la elaboración de la respuesta final.
- Dotar al sistema de capacidad de generalización a colecciones y conjuntos de datos no vistos previamente, mediante la inferencia dinámica del esquema, evitando que la solución quede ligada a una única base de datos.
- Incorporar memoria conversacional, de modo que el asistente pueda mantener el contexto a lo largo de varios turnos de diálogo y resolver referencias anafóricas entre preguntas consecutivas.
- Realizar una evaluación cuantitativa del rendimiento del asistente sobre un *benchmark* propio, diseñado específicamente para medir la corrección funcional de las consultas generadas.

1.4. Alcance y limitaciones

Delimitar el alcance de un sistema de estas características es tan importante como definir sus objetivos, especialmente cuando intervienen consideraciones de fiabilidad y seguridad. Por ello, este trabajo adopta deliberadamente un registro cerrado de tres conjuntos de datos de referencia (*sample_mflix*, *sample_airbnb* y *sample_analytics*) proporcionados por MongoDB Atlas. Esta decisión no es una limitación accidental, sino una elección de diseño, debido a que reducir el universo de datos permite establecer una frontera clara de fiabilidad y validación. Operar sobre datos desconocidos e ilimitados habría diluido esas garantías y habría hecho inviable una evaluación rigurosa.

En el plano del modelo, el sistema se apoya en un LLM ejecutado localmente mediante Ollama. Esta elección responde a dos motivaciones principales: la privacidad, ya que ni las preguntas del usuario ni los datos consultados abandonan el entorno local, y el coste, dado que se elimina por completo la dependencia de APIs de pago. Como contrapartida asumida, el trabajo prescinde del *fine-tuning*. El modelo no se reentrena, sino que se adapta exclusivamente mediante técnicas de prompting. Esta restricción define también el horizonte de lo alcanzable y se discute con detalle en los capítulos de diseño y evaluación. Además, los LLM han demostrado un rendimiento sólido en la generación de consultas en entornos *text-to-SQL* [24, 5], por ello, resulta razonable trasladar el mismo enfoque a la tarea análoga sobre MQL.

2. Estado del arte

La traducción automática de lenguaje natural a consultas estructuradas es uno de los problemas más antiguos y, a la vez, más actuales de la interacción entre las personas y los sistemas de información. Antiguo, porque la aspiración de *preguntarle a la base de datos* con palabras corrientes acompaña a las bases de datos relacionales prácticamente desde su nacimiento, y actual, porque la irrupción de los LLM ha transformado en pocos años un objetivo esquivo en una tecnología utilizable. Este capítulo reconstruye esa trayectoria con el detalle suficiente para situar las decisiones de diseño de MongoMind⁴ dentro de una tradición de investigación, y no como elecciones arbitrarias.

El recorrido se organiza en seis bloques. El primero introduce los fundamentos de los grandes modelos de lenguaje, esto es, la arquitectura Transformer, el preentrenamiento autorregresivo y el escalado, cuyas propiedades sostienen todas las decisiones posteriores. El segundo revisa las interfaces de lenguaje natural a bases de datos (NLIDB) clásicas, basadas en reglas, y explica las razones de fondo por las que nunca alcanzaron un uso general. A continuación, el tercero aborda el campo del *text-to-SQL*, donde se concentran los grandes avances, los *benchmarks* de referencia y, sobre todo, las métricas de evaluación que este trabajo hereda. El cuarto argumenta por qué el salto del mundo relacional (SQL) al documental (NoSQL/MQL) es más difícil y está mucho menos explorado, lo que constituye el hueco que MongoMind viene a ocupar. Los bloques quinto y sexto examinan las dos palancas técnicas que hacen viable el proyecto, la capacidad de los LLM para generar código y el paradigma del *in-context learning* frente al *fine-tuning*.

2.1. Large Language Models

Antes de reconstruir la historia del problema conviene precisar qué son los modelos de lenguaje de gran tamaño y por qué funcionan, pues sus propiedades explican tanto su idoneidad para esta tarea como sus límites, y sobre ellas se apoyan todas las decisiones de diseño que siguen. Un **modelo de lenguaje** es, en esencia, un sistema que aprende una distribución de probabilidad sobre secuencias de texto: dado un fragmento, estima qué continuación es más plausible. Los LLM actuales llevan esa idea a una escala sin precedentes, y de esa escala emergen las capacidades que los hacen útiles.

2.1.1. La arquitectura Transformer

El salto cualitativo se produjo con la arquitectura Transformer [29], que desplazó a las redes recurrentes como base del procesamiento del lenguaje. Su componente central es el **mecanismo de atención** (*self-attention*), que permite a cada elemento de la secuencia *mirar* a todos los demás y ponderar su relevancia, capturando dependencias de largo alcance sin la limitación secuencial de las redes recurrentes (Figura 1). Al prescindir de la recurrencia, el Transformer se paraleliza con eficiencia sobre hardware moderno (GPU), lo que hizo viable entrenar modelos de miles de millones de parámetros sobre corpus de miles de millones de *tokens*. El texto se procesa, precisamente, descompuesto en tokens (unidades subléxicas), y el modelo apila decenas de capas de atención para construir representaciones cada vez más abstractas (Figura 2).

⁴<https://github.com/Brachingo/MongoMind>

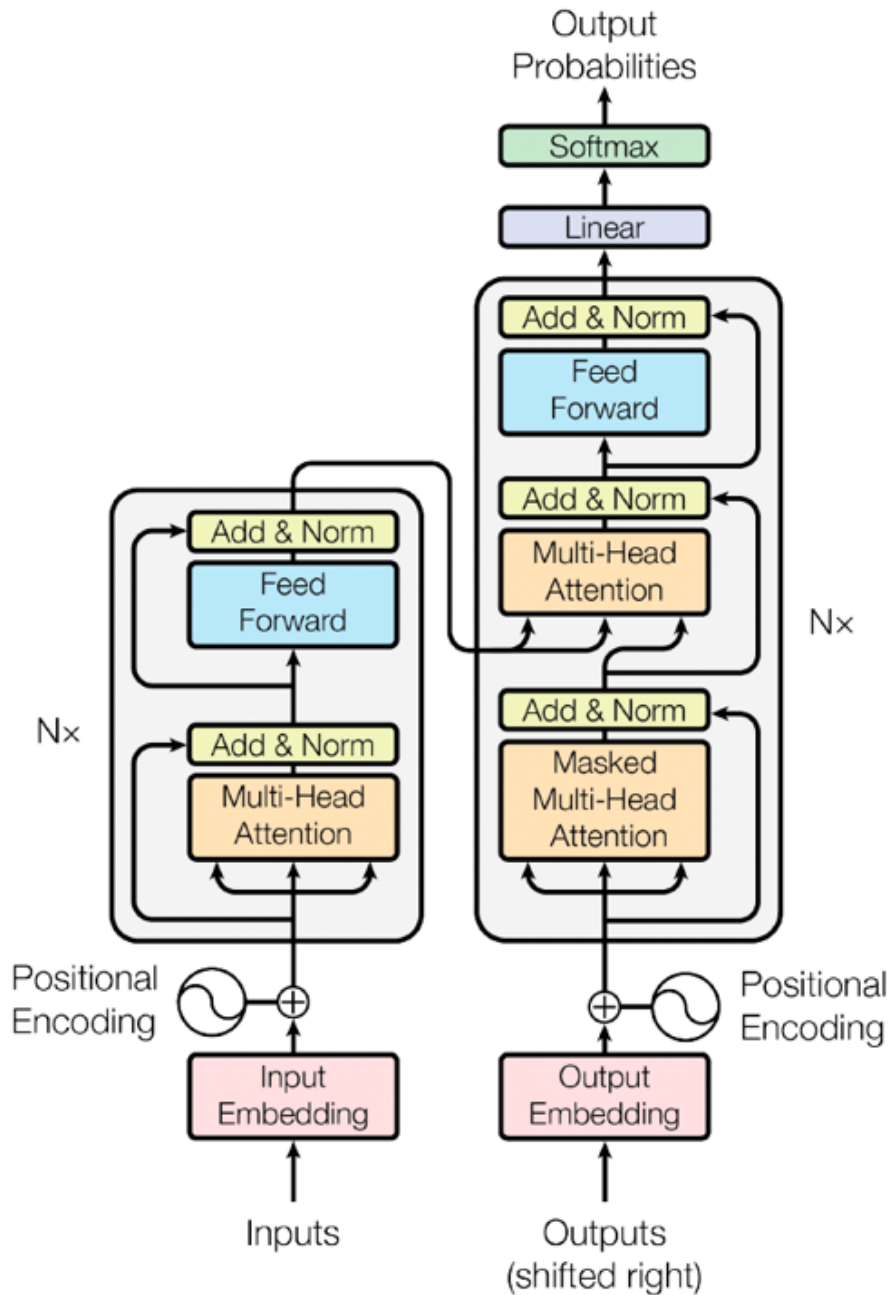


Figura 1: Arquitectura Transformer original. El modelo se compone de un codificador (*encoder*, izquierda) y un decodificador (*decoder*, derecha), ambos formados por una pila de N capas. El codificador combina mecanismos de autoatención multi-cabeza (*Multi-Head Self-Attention*) y redes *Feed Forward*, mientras que el decodificador incorpora además una capa de autoatención enmascarada (*Masked Multi-Head Attention*) para preservar la naturaleza autorregresiva de la generación. Las representaciones de entrada y salida se enriquecen mediante *embeddings* y codificación posicional (*Positional Encoding*), y la predicción final se obtiene a través de una capa lineal seguida de una función *Softmax*. [29]

2.1.2. Preentrenamiento autorregresivo

Los LLM generativos se entrenan de forma autorregresiva y autosupervisada: la tarea consiste, simplemente, en predecir el siguiente token a partir de los anteriores, usando como

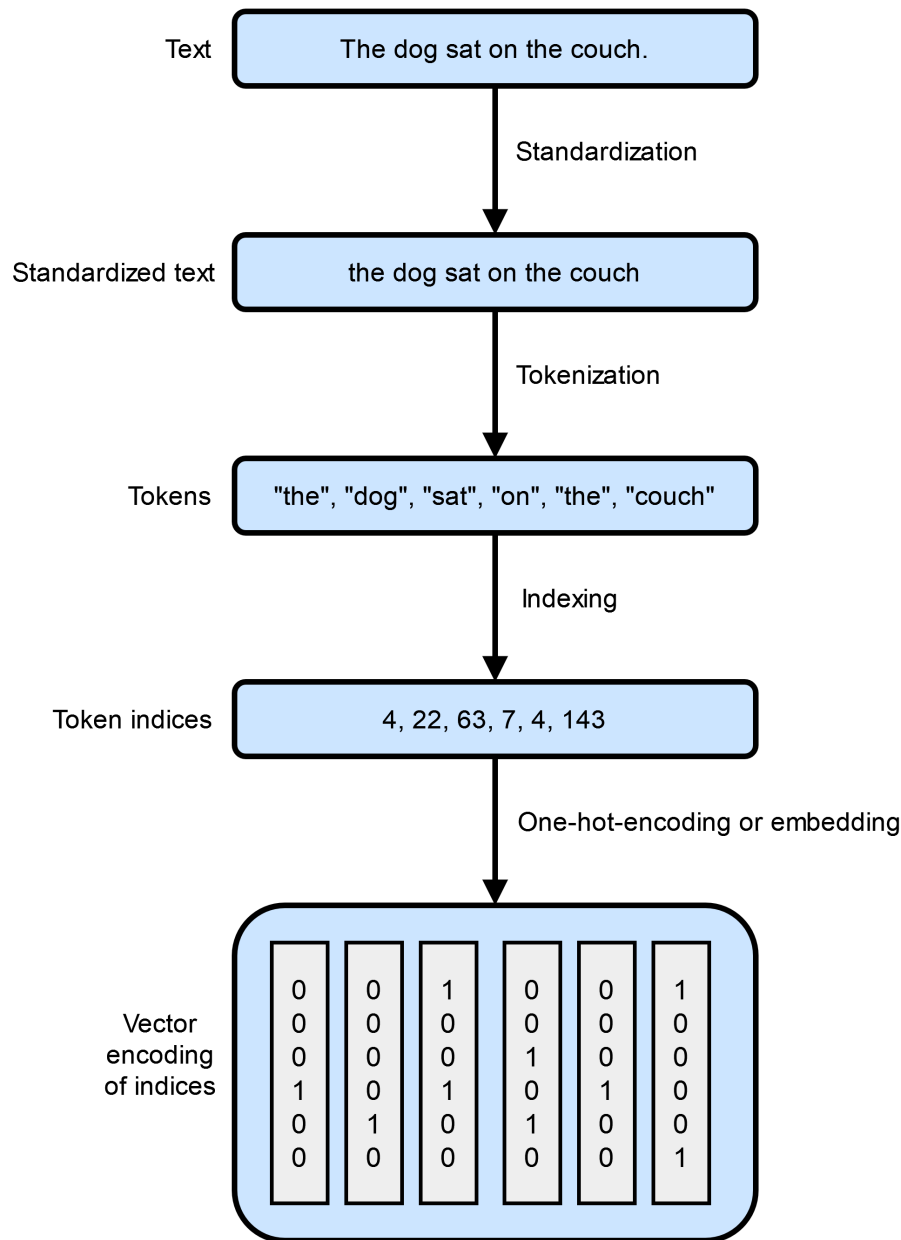


Figura 2: Proceso de tokenización. El texto de entrada se segmenta en unidades discretas (*tokens*) mediante un algoritmo de subpalabras (p. ej., *Byte Pair Encoding*), y cada *token* se asocia a un identificador entero del vocabulario. Estos identificadores constituyen la entrada que el modelo transforma posteriormente en *embeddings*. [7]

etiqueta el propio texto. Esta formulación no requiere anotación humana y, sin embargo, obliga al modelo a interiorizar gramática, hechos, estilos y, hasta cierto punto, patrones de razonamiento. En inferencia, el modelo genera texto token a token, realimentando cada predicción como entrada de la siguiente. La temperatura regula el grado de aleatoriedad de ese muestreo: un valor bajo favorece las continuaciones más probables y, con ello, salidas más deterministas y reproducibles, lo que conviene a una tarea de generación de código donde se busca precisión y no creatividad.

2.1.3. Escalado y capacidades emergentes

El rasgo más determinante de los LLM es que su competencia crece con la **escala**, es decir, cuanto mayor número de parámetros, volumen de datos y cómputo de entrenamiento mejores resultados se obtendrán. A partir de cierto tamaño aparecen capacidades emergentes que no estaban presentes en los modelos menores, entre ellas la que sustenta este trabajo: aprender una tarea nueva a partir de unos pocos ejemplos incluidos en el propio *prompt*, sin reentrenamiento.

2.2. Interfaces de lenguaje natural a bases de datos (NLIDB)

La idea de consultar una base de datos *hablándole* en lenguaje natural es casi tan antigua como las propias bases de datos. Desde los años setenta se construyeron sistemas pioneros que, pese a las limitaciones del hardware y de las técnicas de procesamiento del lenguaje de la época, lograban responder preguntas sobre dominios concretos. El caso más citado es LUNAR[32], un sistema capaz de responder en inglés a preguntas de los geólogos sobre las muestras de roca traídas por las misiones Apolo. A este, le siguieron sistemas como LADDER[8], orientado a consultas militares sobre flotas navales.

El rasgo común de toda esta primera generación es su naturaleza basada en reglas. El sistema disponía de una gramática y de un diccionario semántico escritos a mano por expertos, que mapeaban las construcciones del lenguaje a los elementos del esquema de la base de datos. Mientras la pregunta encajaba en lo previsto, los resultados eran notables; pero bastaba una reformulación inesperada, un sinónimo no contemplado o una construcción sintáctica ligeramente distinta para que el sistema fallara. El paper de referencia [1] documenta con detalle esta época y diagnostica con claridad sus dos grandes males: la fragilidad lingüística (cobertura limitada y comportamiento impredecible fuera de ella) y, sobre todo, la falta de portabilidad. Adaptar uno de estos sistemas a una base de datos nueva no consistía en cambiar una configuración, sino en reescribir buena parte de la gramática y el diccionario semántico, una tarea que requería lingüistas e ingenieros y que podía llevar meses.

Una segunda oleada de sistemas trató de mitigar estos problemas con aproximaciones más sofisticadas. Algunos, como PRECISE[21], optaron por acotar formalmente el problema: en lugar de aspirar a entender cualquier pregunta, delimitaban un subconjunto de preguntas *semánticamente tratables* para las que sí podían garantizar una traducción correcta, renunciando explícitamente al resto. Otros, como NaLIR[10], recuperaron la idea del diálogo de Codd e introdujeron al usuario en el bucle. Así, ante varias interpretaciones posibles, el sistema mostraba sus candidatas y dejaba que la persona eligiera, trasladando la resolución de la ambigüedad al único agente capaz de resolverla sin error. Estos trabajos representan lo mejor de la era antes del aprendizaje profundo (DL, *Deep Learning*) y aportaron ideas que siguen vigentes, pero no resolvieron el problema de fondo en su totalidad, debido a que seguían dependiendo del esquema concreto y costaban mucho de generalizar. Esa barrera mantuvo a las NLIDB confinadas a nichos y demostradores académicos, lejos del uso cotidiano, hasta que el aprendizaje profundo cambió por completo las reglas del juego.

2.3. Text-to-SQL: benchmarks y métricas de evaluación

El punto de inflexión llegó cuando el problema se reformuló como una tarea de *traducción supervisada* abordable con redes neuronales. Este término se refiere al paradigma de entrenamiento donde el modelo aprende a partir de pares de frases ya traducidas por humanos (la frase original y su traducción correcta). El modelo usa esas traducciones de referencia como la etiqueta (de manera similar a un aprendizaje supervisado con *Machine Learning* (ML) o DL) para ajustar sus parámetros y minimizar el error entre lo que predice y la traducción humana. Este giro trajo consigo dos cosas que han marcado el campo hasta hoy: los grandes *benchmarks* públicos y una discusión rigurosa sobre cómo medir el éxito.

El primer gran corpus de uso masivo fue WikiSQL [37], con decenas de miles de pares generados sobre tablas de Wikipedia. Su escala permitió por primera vez entrenar y comparar modelos de forma sistemática, pero su alcance era limitado. Las consultas operaban sobre una única tabla, y seguían patrones relativamente sencillos. El salto cualitativo lo marcó Spider [34], que sigue siendo la referencia del campo. Spider introdujo tres ideas decisivas.

- **Complejidad realista:** sus consultas incluyen uniones entre varias tablas, agregaciones, subconsultas y ordenaciones, reflejando el tipo de preguntas que un analista formula en la práctica.
- **Carácter multidominio y cross-database:** el corpus abarca más de doscientas bases de datos de temáticas dispares.
- **Partición estricta:** quizás la más importante metodológicamente, asegura que las bases de datos del conjunto de test no hayan sido vistas por el modelo durante el entrenamiento. Esta separación obliga a evaluar la capacidad real de generalización del modelo⁵ y no su capacidad de memorizar patrones de bases de datos ya conocidas.

Spider tuvo además descendencia en la dirección que más interesa a MongoMind: la conversacional. Sus variantes SParC y CoSQL [35, 33] extendieron el problema al diálogo multi-turno, donde una pregunta no se interpreta de forma aislada sino en el contexto de las anteriores, lo que exige resolver referencias anafóricas (p. ej., *y de esas, ¿cuáles...?*, *ordénalas por...*). Más recientemente, BIRD [12] ha vuelto a elevar el listón al incorporar bases de datos de gran tamaño sin curar (valores faltantes, inconsistencias, ruido...) y al introducir la eficiencia de ejecución como una dimensión evaluable más, no solo la corrección.

De esta tradición proceden las dos métricas que vertebran cualquier evaluación seria en este ámbito, y que conviene distinguir con cuidado porque su elección no es neutral:

2.3.1. Exact match accuracy

La coincidencia exacta compara la consulta generada con la de referencia a nivel de texto, normalmente tras normalizar componentes (alias, orden de las cláusulas, mayúsculas). Es fácil de calcular y no requiere ejecutar nada, pero cuenta con un defecto grave: penaliza como incorrectas consultas que en realidad son correctas pero están escritas de otra forma.

⁵Su habilidad para enfrentarse a un esquema nuevo.

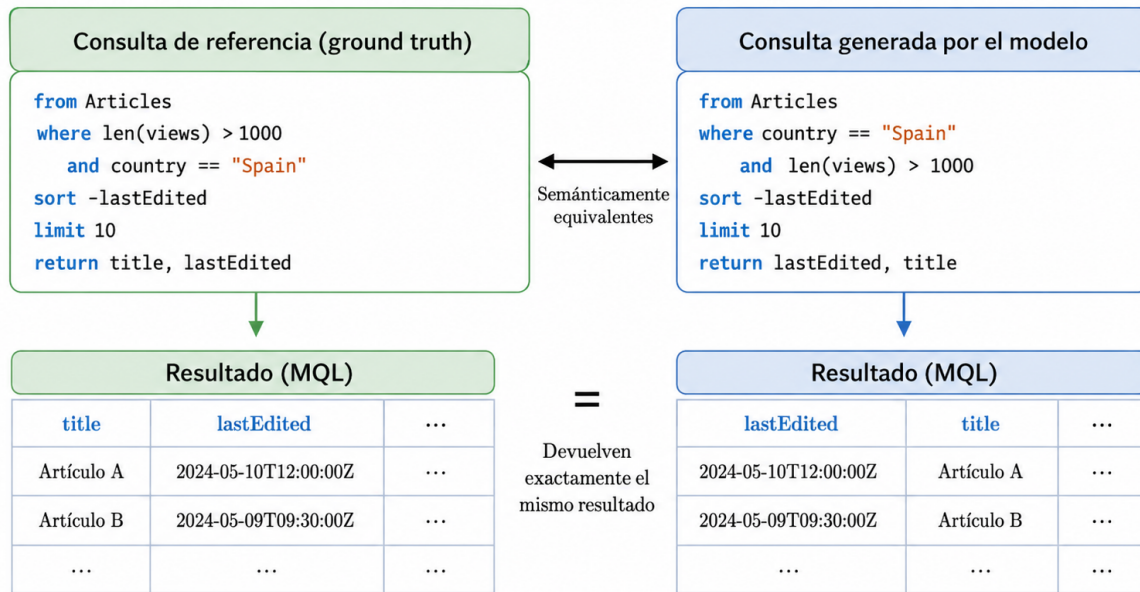


Figura 3: Ejemplo de consultas semánticamente equivalentes (pseudocódigo).

Dos consultas pueden devolver exactamente el mismo resultado y, sin embargo, diferir en el orden de las condiciones, en el nombre de una columna proyectada o en el uso de un operador equivalente. La coincidencia exacta las considera distintas, infravalorando sistemáticamente al modelo (Figura 3).

2.3.2. Execution accuracy

La corrección por ejecución resuelve este problema ejecutando ambas consultas contra la base de datos y comparando sus resultados. Si los resultados coinciden, la consulta se da por correcta, con independencia de cómo esté escrita. Esta métrica capta la equivalencia funcional, que es lo que de verdad le importa al usuario. De esta manera, también se resuelve el problema principal de *exact match accuracy*. Su único coste es operativo, ya que hace falta una base de datos poblada, y ejecutar las consultas.

El consenso del campo, consolidado en Spider y reforzado en BIRD, es que la corrección por ejecución es la métrica más fiel a la utilidad real del sistema. MongoMind hace suya esta conclusión y la adopta como métrica principal. No compara cadenas de texto, sino que ejecuta tanto la consulta generada como la de referencia y compara sus resultados, con estrategias de comparación adaptadas a cada tipo de salida (recuentos, escalares, conjuntos de documentos o agrupaciones).

En el plano de los modelos, la trayectoria reciente es igualmente instructiva. Antes de los LLM, el estado del arte lo marcaban arquitecturas especializadas como RAT-SQL[30], que codificaba explícitamente las relaciones del esquema (tablas, columnas, claves) para guiar la generación, o PICARD[26], que restringía la decodificación del modelo para garantizar que la salida fuera SQL sintácticamente válido. Eran soluciones potentes pero muy específicas, costosas de entrenar y difíciles de trasladar a otros dominios.

La llegada de los LLM generalistas cambió el panorama por completo. Se demostró que un modelo de propósito general sin entrenamiento específico para la tarea y guiado únicamente mediante prompting alcanzaba resultados competitivos en Spider [24]. A partir de ahí, las mejores aproximaciones dejaron de centrarse en arquitecturas a medida y pasaron a centrarse en cómo formular el problema al modelo: descomponer la consulta en pasos, seleccionar buenos ejemplos y permitir la autocorrección, como en DIN-SQL[22]. Este desplazamiento de la arquitectura al prompt sienta los cimientos directos sobre los que se construye MongoMind.

2.4. Del SQL al NoSQL/MQL

Todo lo anterior (benchmarks, métricas, modelos...) se ha desarrollado de forma abrumadora en torno a SQL y al modelo relacional. Es comprensible, SQL es un lenguaje maduro, estandarizado y omnipresente, y existen corpus abundantes para entrenarlo y evaluarlo. Pero el ecosistema de datos actual es mucho más heterogéneo, y una parte muy significativa de la información vive en bases de datos NoSQL, entre las que MongoDB ocupa una posición destacada. Trasladar el problema *text-to-query* a MongoDB Query Language (MQL) va más allá de una simple adaptación. Esta nueva implementación introduce nuevas capas de dificultad que conviene exponer con detalle, porque justifican buena parte de las decisiones de diseño e implementación del proyecto.

2.4.1. Documentos anidados, heterogéneos y sin esquema impuesto

A diferencia de una base de datos relacional, en la que el esquema (tablas, columnas y tipos) se declara de antemano y resulta de obligado cumplimiento, MongoDB almacena documentos JSON/BSON⁶ bajo un esquema flexible (Figura 4). Dos documentos de una misma colección pueden presentar campos distintos, un campo puede estar ausente en unos y presente en otros, y las estructuras pueden anidarse a profundidad arbitraria, alojando subdocumentos dentro de subdocumentos. En `sample_mflix`, por ejemplo, una película contiene campos anidados como `imdb` (con `rating`, `votes` e `id`) o `tomatoes`, así como arrays del tipo `cast`, `genres` o `directors`. La consecuencia práctica es profunda: no existe un catálogo del que el sistema pueda partir. Mientras que un sistema *text-to-SQL* puede leer el esquema directamente del diccionario de datos, un sistema $NL \rightarrow MQL$ debe inferirlo por muestreo, examinando un subconjunto de documentos para deducir qué campos existen, de qué tipo son y con qué frecuencia aparecen.

2.4.2. Tipado heterogéneo dentro de un mismo campo

La problemática anterior se agrava por el hecho de que la flexibilidad de MongoDB permite que un mismo campo contenga valores de tipos distintos en documentos distintos. No se trata de un supuesto hipotético. En `sample_mflix`, campos como `imdb.rating` o `runtime`, que esperaríamos como campos numéricos, contienen en algunos documentos la cadena vacía. Una consulta que ordene por `imdb.rating` o que calcule su media fallará o devolverá resultados espurios si no contempla esto. La solución se encuentra en la introducción de guardas de tipo explícitas, como `{"$type": "number"}`, en las consultas, constituyendo así una de las lecciones de implementación más características de este trabajo y ejemplifica con claridad una dificultad que sencillamente no existe en el ámbito relacional, donde el tipo de una columna está garantizado.

⁶JavaScript Object Notation / Binary JSON

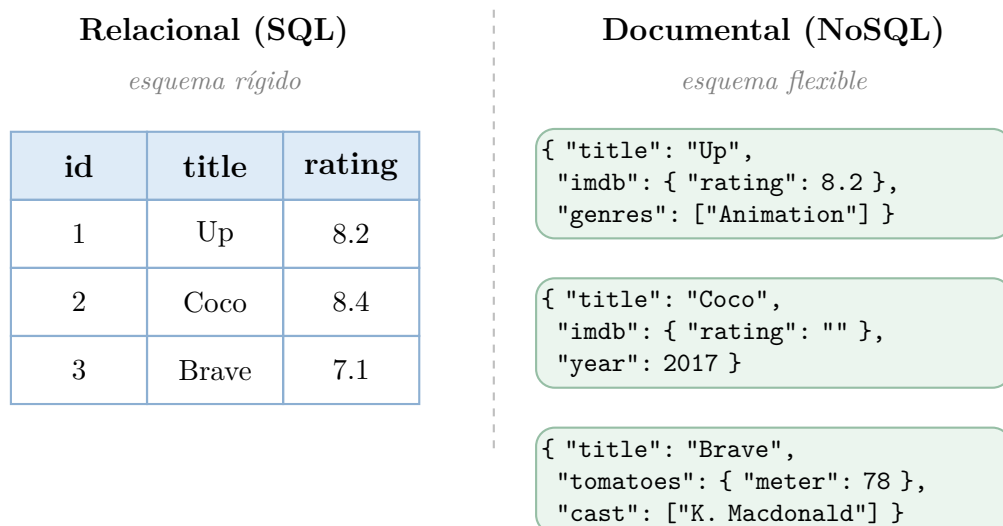


Figura 4: La base relacional impone un esquema rígido declarado de antemano (columnas y tipos uniformes en todas las filas), mientras que la documental admite un esquema flexible, con campos heterogéneos, ausencias y estructuras anidadas dentro de una misma colección.

2.4.3. El *aggregation pipeline*

En SQL, una consulta no trivial sigue siendo una única sentencia declarativa con cláusulas bien delimitadas (`SELECT ... FROM ... WHERE ... GROUP BY ... ORDER BY`). En MongoDB, por el contrario, las consultas complejas se expresan mediante un *aggregation pipeline*: una secuencia ordenada de etapas (`$match`, `$group`, `$unwind`, `$sort`, `$project`, `$lookup`, etc.) en la que la salida de cada una alimenta a la siguiente. Esto plantea dos dificultades para un generador automático:

1. El orden de las etapas resulta significativo y debe razonarse explícitamente, ya que filtrar antes o después de agrupar conduce a resultados distintos.
2. El tratamiento de los arrays exige una atención especial. Para agrupar o contar por los elementos de un array suele ser necesario desplegarlo previamente mediante una etapa `$unwind`, un paso que carece de equivalente conceptual directo en SQL y que los modelos tienden a omitir si no se les guía de forma expresa. De nuevo, esta es una de las indicaciones que MongoMind incorpora a sus plantillas *few-shot*.

2.4.4. Ausencia de un benchmark canónico

Quizá la consecuencia más relevante para un trabajo de investigación sea que, frente a la riqueza de recursos del mundo SQL (Spider, BIRD, WikiSQL y muchos otros), no existe un *benchmark* consolidado y ampliamente adoptado para la tarea $NL \rightarrow MQL$. No se dispone de un equivalente a Spider sobre el que evaluar el rendimiento y establecer comparaciones con otros trabajos. Esta carencia presenta una doble vertiente. Por un lado, supone un obstáculo metodológico, pues obliga a construir un *benchmark* propio desde cero, con el consiguiente esfuerzo de diseño, redacción y verificación. Por otro, y de mayor calado, evidencia que el dominio $NL \rightarrow MQL$ se encuentra comparativamente inexplorado, lo que transforma la dificultad en oportunidad: existe un margen real de contribución. Esta constatación constituye la principal motivación de MongoMind y aconseja acompañarla de una revisión actualizada de la literatura más reciente, dado el ritmo al que avanza el campo.

En síntesis, transitar de SQL a MQL no consiste en cambiar de dialecto, sino en afrontar un problema dotado de menor estructura de partida (esquema inferido en lugar de declarado), datos más ruidosos (tipado heterogéneo) y un lenguaje de consulta más procedimental y verboso (el *pipeline*). MongoMind asume estas dificultades como parte central de su planteamiento.

2.5. LLM para generación de código y consultas

Generar MQL es, en última instancia, un caso particular de generación de código, producir una expresión formal, sintácticamente correcta y semánticamente adecuada, a partir de una descripción en lenguaje natural. Conviene por tanto situar el trabajo en el contexto de lo que los LLM han logrado en este terreno más amplio, porque de ahí se extraen tanto la confianza en la viabilidad del enfoque como sus límites.

El hito fundacional fue Codex [3], el modelo derivado de GPT que dio lugar a *GitHub Copilot*. Codex demostró que un modelo de lenguaje entrenado sobre grandes cantidades de código podía traducir descripciones en lenguaje natural a programas funcionales en lenguajes como Python con una tasa de acierto notable. Igual de importante que el modelo fue el benchmark que lo acompañó, HumanEval, y la métrica que popularizó, *pass@k*. En lugar de comprobar si el código generado se parece a una solución de referencia, se ejecuta contra una batería de pruebas y se considera correcto solo si las supera. Es, de nuevo, la filosofía de la evaluación funcional, que juzga el código por lo que hace. Tras Codex llegaron sistemas que ampliaron la frontera, como AlphaCode[13], capaz de competir en problemas de programación competitiva, o modelos abiertos especializados como Code Llama[28, 6], que pusieron capacidades de generación de código de calidad al alcance de cualquiera sin depender de una API comercial.

2.6. In-context learning y few-shot prompting frente a *fine-tuning*

Si hay un avance que explica por qué un proyecto como MongoMind es viable sin grandes recursos de cómputo ni datos de entrenamiento etiquetados, ese es el *in-context learning*. Un modelo de lenguaje suficientemente grande puede resolver tareas nuevas a partir únicamente de unos pocos ejemplos incluidos en el propio *prompt*, sin modificar ni uno solo de sus parámetros [2]. Basta con mostrarle en el contexto unas cuantas demostraciones de la tarea (de ahí el nombre *few-shot*) para que el modelo infiera el patrón y lo aplique a una entrada nueva. La adaptación, que antes exigía reentrenar, pasa a ocurrir en tiempo de inferencia, simplemente redactando bien la entrada.

Este hallazgo dio origen a toda una disciplina, el *prompt engineering*, dedicada a estudiar cómo formular las entradas para sacar el máximo partido del modelo. Se ha investigado, por ejemplo, qué papel juegan exactamente las demostraciones y por qué funcionan, y se han desarrollado técnicas para problemas que requieren razonamiento intermedio, como el chain-of-thought [31], que mejora los resultados pidiendo al modelo que explicita los pasos antes de dar la respuesta final. Para MongoMind, lo relevante es que la calidad del sistema depende en gran medida de la calidad del contexto que se le proporciona al modelo, siendo este principalmente el esquema de la colección y, sobre todo, los ejemplos (pregunta → MQL) cuidadosamente seleccionados y verificados.

La alternativa al in-context learning es el *fine-tuning*: especializar el modelo en la tarea reentrenando sus parámetros con un conjunto de ejemplos. Hoy esta vía es más accesible que nunca gracias a técnicas de ajuste eficiente como LoRA [9], que permiten adaptar un modelo grande modificando solo una pequeña fracción de sus pesos. La decisión entre *fine-tuning* y *prompting* es una de las elecciones de diseño centrales de este proyecto, y debe justificarse con cuidado:

- El *fine-tuning* puede ofrecer una mayor precisión en la tarea concreta, pero tiene un coste alto en este contexto. Primero, se exige un conjunto de entrenamiento etiquetado de tamaño considerable, pero este no existe para NL→MQL. Segundo, requiere recursos de cómputo (GPU) altos en LLMs. Y tercero, el modelo ajustado queda especializado en lo que ha visto, de modo que dar soporte a una colección o a un dataset nuevo implicaría, en el peor caso, reentrenar. Esto reproduce exactamente el problema de portabilidad que condenó a las NLIDB clásicas.
- El *few-shot prompting*, en cambio, permite adaptar el sistema a un esquema nuevo simplemente cambiando el contexto. Basta con proporcionar el esquema de la nueva colección y unos ejemplos representativos para que el mismo modelo, sin tocar sus pesos, empiece a generar MQL para ella. La adaptación se vuelve barata, rápida y sin GPU.

MongoMind adopta deliberadamente la segunda vía. Su principio rector es que el LLM es el único componente de aprendizaje del sistema y toda la adaptación ocurre vía *prompt*, mediante el esquema (estático o inferido) y las plantillas *few-shot* por colección. Gracias a ella el sistema generaliza a colecciones y datasets no contemplados en el diseño original sin reentrenamiento alguno, cumpliendo así uno de los objetivos del trabajo. En este proyecto, mejorar las plantillas *few-shot*, añadiendo un bloque de reglas explícitas y un puñado de ejemplos verificados orientados a los errores más frecuentes, elevó la corrección funcional en el conjunto de desarrollo del 73,9 % al 89,1 %, una mejora sustancial conseguida sin modificar el modelo, solo su contexto, como se verá más adelante en la sección de resultados.

3. Análisis del problema y requisitos

Definido el contexto y el estado del arte, esta sección traduce los objetivos generales del trabajo en requisitos concretos. El propósito es delimitar qué debe hacer el sistema, bajo qué restricciones, y sentar las bases sobre las que se evaluará. Se distinguen tres categorías de requisitos: funcionales, no funcionales y de seguridad. Esta separación tiene aquí especial sentido, porque en un sistema que genera y ejecuta código a partir de texto libre del usuario la seguridad no es una característica más, sino una condición de viabilidad.

3.1. Análisis del problema

El problema central que MongoMind aborda puede enunciarse en una frase: traducir una pregunta formulada en lenguaje natural (en español o en inglés) a una consulta MQL válida, ejecutable y segura, ejecutarla contra una base de datos MongoDB y devolver el resultado de forma comprensible, sin exigir al usuario ningún conocimiento de MQL. Detrás de esta formulación se esconde una cadena de subproblemas, cada uno de los cuales da lugar a un módulo del sistema. El siguiente cuadro los resume (Cuadro 1).

Nº	Subproblema	Descripción
1	Comprensión de la intención y enrutado	Antes de generar nada, el sistema debe determinar a qué colección se refiere la pregunta. Incluso dentro de <i>sample_mflix</i> , podría tratarse de <i>movies</i> , <i>comments</i> , <i>theaters</i> o <i>users</i> , porque el esquema y los ejemplos que se dan al modelo dependen de ello. En una conversación, si una pregunta de seguimiento no menciona colección, se continúa sobre la anterior.
2	Generación del MQL	A partir del esquema y de ejemplos, el sistema produce la consulta, que en MongoDB puede ser un find (un objeto de filtro) o un <i>aggregation pipeline</i> (una lista de etapas, para agrupaciones, cálculos o uniones). Debe ser sintácticamente válida y semánticamente fiel a la pregunta.
3	Validación y ejecución	La salida de un LLM puede no ser JSON válido o contener operaciones peligrosas. Antes de tocar la base de datos, el sistema garantiza que el MQL es JSON bien formado y está libre de operaciones de escritura, y solo entonces lo ejecuta.
4	Devolución del resultado	El resultado se devuelve de forma legible, mostrando además el propio MQL generado para que pueda inspeccionarse.

Cuadro 1: Subproblemas que componen el pipeline de MongoMind, del lenguaje natural al resultado.

A estos cuatro subproblemas, que forman un *pipeline* lineal, se añaden dos dimensiones transversales. La conversacional, donde el usuario puede encadenar preguntas que se refieran a las anteriores (p. ej., *¿y ordenadas por año?*), lo que obliga a recordar el contexto reciente y resolver las referencias anafóricas. Y la multi-dataset, que el sistema pueda operar sobre varios conjuntos de datos, lo que pone a prueba su capacidad de generalización.

3.2. Requisitos funcionales

Los requisitos funcionales describen las capacidades observables que el sistema ofrece al usuario. En primer lugar, el sistema traduce a MQL preguntas formuladas tanto en español como en inglés, sin que el usuario indique el idioma, y ejecuta el MQL resultante contra MongoDB Atlas para devolver los documentos reales que satisfacen la consulta. El resultado se presenta de forma legible y, cuando procede, se acompaña de un mensaje informativo, distinguiendo por ejemplo una consulta correcta sin resultados de un error real de ejecución.

Para servir varias colecciones, el sistema enruta cada pregunta a la colección adecuada del dataset activo por coincidencia de palabras clave, y reutiliza la colección de la pregunta anterior cuando una de seguimiento no aporta indicios. Da servicio a las distintas colecciones del dataset (p. ej., `movies`, `comments`, `theaters` y `users` en *sample_mflix*) mediante plantillas few-shot específicas cuando existen; para las colecciones sin plantilla propia, infiere el esquema por muestreo en tiempo de ejecución y construye el contexto del *prompt* de forma dinámica, lo que le permite atender colecciones no previstas en el diseño.

El sistema opera además sobre un registro cerrado de datasets conocidos (*sample_mflix*, *sample_airbnb* y *sample_analytics*), seleccionables desde la interfaz; al cambiar de dataset se reinicia la memoria conversacional, ya que las referencias anafóricas entre datasets distintos no tienen sentido. Esa memoria conserva una ventana de los últimos intercambios de cada sesión para resolver las referencias en las preguntas de seguimiento. Por transparencia, el MQL generado se muestra siempre junto a los resultados, de modo que el usuario o un revisor técnico pueda auditar exactamente qué consulta se ejecutó. Toda esta interacción se ofrece a través de una aplicación web de chat.

3.3. Requisitos no funcionales

Los requisitos no funcionales no describen funciones concretas, sino cualidades transversales del sistema. La más característica es el coste cero de API. Toda la inferencia se realiza con un LLM local mediante Ollama, sin ninguna llamada de pago a servicios en la nube. De ello se deriva la privacidad, pues las preguntas, el esquema y los datos consultados no abandonan el entorno local de ejecución. La interacción debe además resultar fluida, con una latencia de pocos segundos por consulta (del orden de 1,27 s de media con el modelo principal en el *benchmark*).

El sistema busca también ser reproducible. El modelo es de versión fija y la generación se realiza a temperatura baja (0,1) para acotar la variabilidad y favorecer la reproducibilidad de cada evaluación. Como los modelos locales emiten a veces JSON *casi válido*, el sistema repara los defectos estructurales más habituales (claves sin comillas, comas finales) y reintenta el parseo antes de rendirse. Las respuestas se limitan por defecto a 100 documentos para no saturar la interfaz ni la red. Por último, la arquitectura es modular. Añadir un dataset se reduce a una nueva entrada en el registro, y dar soporte a una colección, a una plantilla de ejemplos guiados. Además, el sistema se acompaña de pruebas unitarias y de integración, con un benchmark verificado por completo como ejecutable contra la base de datos real.

3.4. Requisitos de seguridad

Los requisitos de seguridad merecen un tratamiento como requisitos duros, no negociables, porque MongoMind genera código y lo ejecuta a partir de texto libre del usuario. Sin las protecciones adecuadas, podría ser inducido a ejecutar consultas que modifiquen o destruyan datos, sobrecarguen el servicio o filtren información. Por ello se adopta un enfoque de defensa en profundidad, con varias barreras independientes de modo que el fallo de una no comprometa el conjunto.

La barrera última es la conexión de solo lectura: en producción, la conexión a MongoDB usa un usuario sin permisos de escritura, de manera que la propia base de datos rechazaría cualquier intento de modificación. Antes de llegar ahí, el generador valida el MQL y rechaza cualquier operación de escritura o destructiva (`insertOne`, `updateMany`, `deleteMany`, `drop` u operadores como `$out` y `$merge`), provocando un fallo controlado que nunca toca la base de datos. Además, el MQL se parsea y valida como JSON bien formado antes de pasar al conector: en ningún caso se entrega a PyMongo texto crudo del LLM, sino estructuras de datos ya validadas.

La entrada del usuario también se controla: cada pregunta se normaliza en Unicode, se le retiran caracteres de control, se colapsan los espacios y se imponen límites de longitud (entre 2 y 500 caracteres) antes de incorporarla al *prompt*. Un limitador de ventana deslizante restringe las peticiones por cliente (20 peticiones por minuto y por dirección IP) y responde con un rechazo (HTTP 429) a las que exceden el límite. Cada consulta ejecutada se registra como una línea estructurada para poder auditar el uso *a posteriori*. Finalmente, el sistema opera únicamente sobre un registro cerrado de datasets, no se expone una base de datos arbitraria, sino un dominio acotado, verificado y bajo control. En conjunto, estas medidas reflejan una convicción de fondo: un asistente NL→MQL solo es responsable si trata su propia salida como potencialmente insegura por defecto y la valida antes de actuar sobre datos reales.

4. Conjuntos de Datos

La selección de los datos busca un equilibrio entre realismo (datos lo bastante complejos para que el problema sea significativo) y control (un dominio acotado y verificable). Todos los conjuntos empleados son *sample datasets* oficiales de MongoDB Atlas [16, 15], alojados y consultados directamente en la nube de Atlas en lugar de en una instancia local. Esta decisión responde a varias razones. En primer lugar, son conjuntos públicos, versionados y mantenidos por MongoDB, lo que garantiza que cualquier lector pueda reproducir el entorno de evaluación cargándolos en su propio *cluster* con un esquema idéntico al utilizado aquí. En segundo lugar, operar contra Atlas sitúa al sistema en un escenario realista de despliegue en lugar de un entorno de juguete, lo que da mayor validez a las medidas de latencia y a las garantías de seguridad (como la conexión de solo lectura). Atlas ofrece estos datasets ya poblados y con índices representativos, utilizarlos también evita el sesgo de construir datos a medida del propio sistema.

Como conjunto de referencia se elige *sample_mflix* [19], sobre el que se construyen el *benchmark* y la evaluación principal. Es un conjunto bien documentado y de complejidad realista: combina campos anidados (como `imdb` o `tomatoes`), *arrays* (`cast`, `genres`, `directors`) y, de forma muy oportuna, el tipado heterogéneo de campos como `imdb.rating` o `runtime`, que a veces son numéricos y a veces cadena vacía. Es además lo bastante rico como para cubrir todo el espectro de consultas a evaluar, desde filtros simples hasta agregaciones con `$unwind`, `$group` y `$lookup`. Modela un catálogo de cine repartido en varias colecciones: `movies`, la colección por defecto, contiene las películas con sus metadatos (título, año, géneros, reparto, dirección y puntuaciones de IMDb y Rotten Tomatoes); `comments` almacena los comentarios de los usuarios sobre las películas, vinculados a estas por su identificador, lo que la hace idónea para probar uniones mediante `$lookup`; `theaters` recoge salas de cine con su ubicación geográfica; y `users` guarda las cuentas de usuario con sus credenciales (Figura 5).

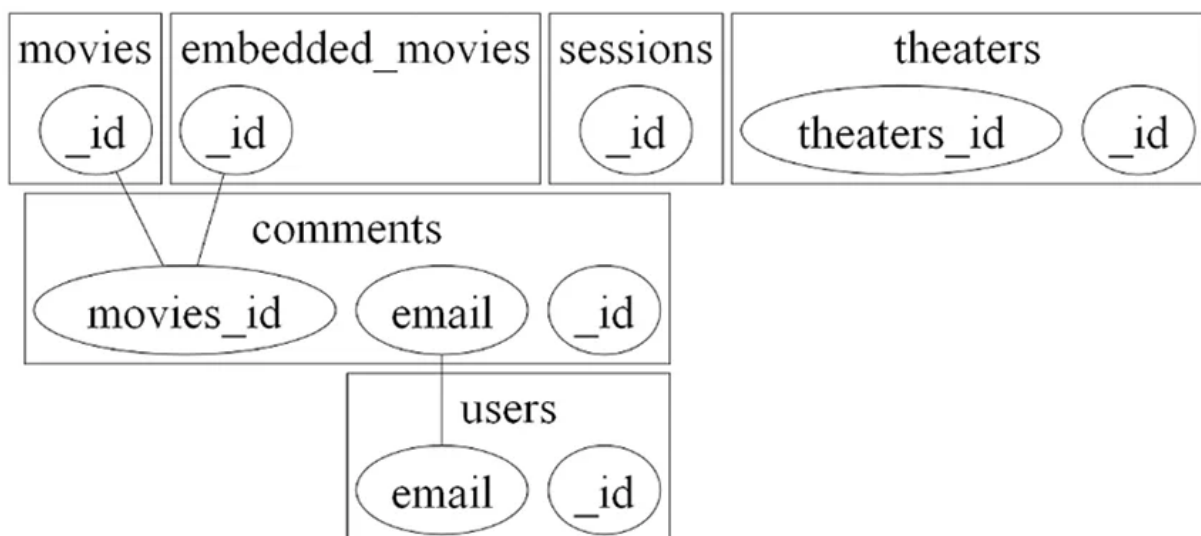


Figura 5: Grafo de relaciones del dataset *sample_mflix*, con sus cuatro componentes desconectados y los vínculos entre colecciones [36].

Para evaluar la generalización se incorporan dos datasets adicionales del catálogo de Atlas. *sample_airbnb* [17] modela anuncios de alojamiento, y su colección principal (**listingsAndReviews**) reúne en un mismo documento los datos del alojamiento (precio, ubicación, servicios y descripciones extensas) junto con sus reseñas embebidas. Por otro lado, *sample_analytics* [18] representa datos financieros de clientes a través de tres colecciones: **customers**, **accounts** y **transactions**, donde esta última agrupa las operaciones en *arrays* dentro de cada documento (*bucketed*), lo que exige un `$unwind` para operar sobre las transacciones individuales.

El conjunto se mantiene deliberadamente cerrado en tres datasets. Esta decisión, ya justificada por la seguridad, responde también al alcance del trabajo. El objetivo no es construir un conector universal capaz de enfrentarse a cualquier base de datos MongoDB (un objetivo demasiado ambicioso y poco fiable), sino un asistente fiable, verificable y seguro sobre un dominio acotado, que demuestre la validez del enfoque y sienta las bases para futuras ampliaciones.

5. Diseño de la solución

Las secciones anteriores han establecido qué debe hacer el sistema y por qué las decisiones de fondo (LLM local, adaptación por *prompt*, evaluación funcional) se justifican desde el estado del arte. Este capítulo da el paso del «qué» al «cómo». Describe la arquitectura que materializa esos requisitos, el reparto de responsabilidades entre módulos y los contratos de entrada y salida que los conectan. La exposición se mantiene en el plano del diseño, dejando los detalles concretos de codificación para la siguiente sección.

El principio rector de todo el diseño es la simplicidad disciplinada. En lugar de un sistema con componentes que se llaman entre sí de forma cruzada, MongoMind se organiza como una secuencia lineal de etapas, cada una con una única responsabilidad bien delimitada, de modo que la salida de una alimenta a la siguiente sin ramificaciones ni estados compartidos ocultos. Se ha optado por esta arquitectura porque una estructura lineal y modular es más fácil de razonar, de probar y de extender.

5.1. Visión general y arquitectura

El núcleo de MongoMind es un *pipeline* de cuatro etapas que transforma una pregunta en lenguaje natural en una respuesta basada en datos reales. El recorrido de una pregunta es el siguiente:

1. Comprensión y enrutado: la pregunta, ya sanitizada, se analiza para determinar a qué colección del dataset activo se refiere. El resultado es el nombre de una colección.
2. Generación del MQL: con la colección decidida, se construye el contexto del modelo y se hace la llamada al LLM, cuya salida se valida como JSON y se comprueba que no contenga operaciones de escritura. El resultado es una consulta MQL (un `dict` para un `find` o una `list` para un *aggregation pipeline*).
3. Ejecución: la consulta validada se ejecuta contra MongoDB Atlas mediante PyMongo y devuelve los documentos que la satisfacen, acotados a un máximo configurable.
4. Presentación: los resultados se formatean y se devuelven al usuario junto con el propio MQL generado.

La estructura del proyecto refleja con fidelidad este reparto. Bajo `src/core/` residen los módulos de lógica pura: `nlp`, `datasets`, `mql_generator`, `schema_inferencer`, `db_connector`, más los de seguridad. Bajo `src/web/` vive la capa de presentación, y en `src/prompts/templates/` se guardan las plantillas *few-shot* ordenadas por colección. Esta separación entre lógica y presentación es la que permite que toda la cadena de generación se ejerza desde las pruebas y la evaluación sin necesidad de levantar la interfaz web.

Es relevante mencionar dos propiedades de diseño del proyecto. La primera, es que el *pipeline* de MongoMind es estrictamente unidireccional, de modo que ningún módulo posterior puede modificar el estado de uno anterior, lo que elimina la posibilidad de errores de acoplamiento. La segunda propiedad, es que cada etapa tiene una función explícita y verificable, todas tienen una firma de entrada y un tipo de salida bien definidos (Figura 6).

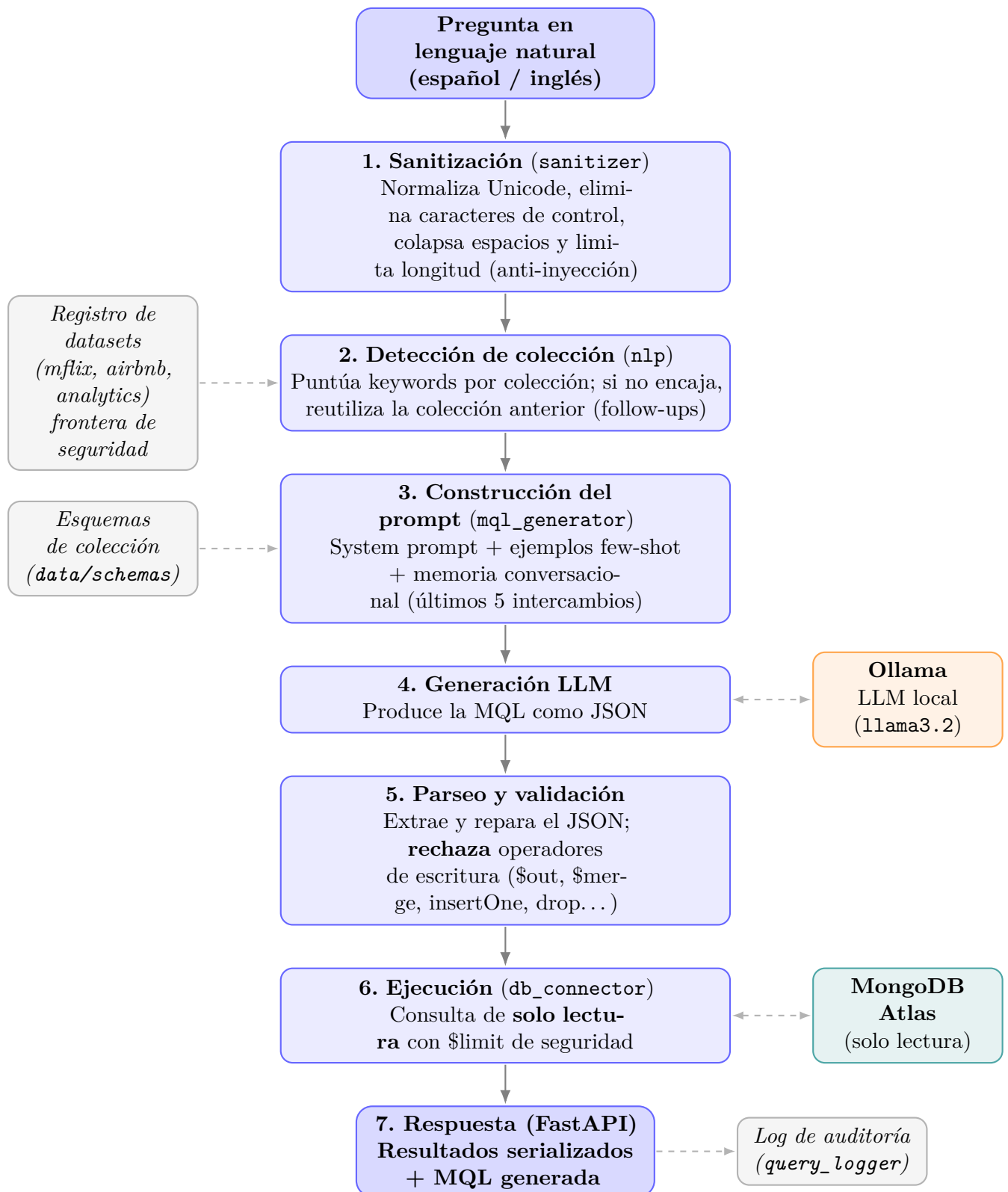


Figura 6: Arquitectura del *pipeline* de MongoMind: traducción de lenguaje natural a MQL (NL→MQL), validación de solo lectura y ejecución contra MongoDB Atlas mediante un LLM local servido con Ollama.

5.2. Módulos del *core*

El módulo central de MongoMind no es una pieza monolítica, sino un conjunto de módulos, cada uno con una única responsabilidad bien delimitada. La idea de fondo es sencilla: si cada parte hace una sola cosa y se relaciona con las demás a través de un acuerdo claro sobre qué recibe y qué entrega, el sistema se puede entender, probar y modificar por partes sin que un cambio en un módulo implique efectos imprevistos en otro.

El primero de ellos es el registro de datasets, que funciona como el mapa del dominio. Ahí se recoge, en un único sitio, todo lo que caracteriza a cada base de datos que el sistema sabe consultar, incluyendo qué colecciones contiene, cuál es la que se usa por defecto y qué palabras suelen delatar que una pregunta se refiere a una u otra. Concentrar esta información en un solo lugar tiene una ventaja práctica inmediata, ya que añadir soporte para una base de datos nueva se reduce, casi siempre, a ampliar este registro, sin tocar el resto del sistema.

Sobre ese mapa trabaja el módulo de comprensión y enrutado, cuya misión es decidir a qué colección apunta cada pregunta. Compara la pregunta con las palabras clave de cada colección y se queda con la que mejor encaja, sin recurrir a ningún modelo de aprendizaje adicional. El único componente *inteligente* del sistema es el modelo de lenguaje, y todo lo demás se mantiene predecible. El enrutado incorpora además un detalle pensado para la conversación real, de manera que cuando una pregunta no contiene ninguna pista evidente (como ocurre en los seguimientos del tipo *¿y solo las del año pasado?*), se asume que seguimos hablando de la misma colección que en la pregunta anterior (Figura 7).

En el centro de todo está el generador de consultas, la pieza que convierte el lenguaje natural en una consulta de MongoDB. Es el módulo que coordina la traducción, reúne las instrucciones y los ejemplos adecuados para la colección en cuestión, añade el hilo reciente de la conversación, plantea la pregunta al modelo de lenguaje y sanea la respuesta antes de darla por buena. Su compromiso es estricto y en eso reside buena parte de su fiabilidad, ya que nunca devuelve el texto en bruto del modelo, sino únicamente una consulta ya comprobada y lista para ejecutarse, y si la respuesta resulta irrecuperable o insegura, prefiere fallar de forma controlada antes que arriesgar.

Cuando una colección no dispone de ejemplos preparados a mano, entra en juego el inferidor de esquema. En lugar de quedarse sin contexto, este módulo observa una pequeña muestra de documentos reales y deduce por sí mismo cómo es la colección, qué campos tiene, de qué tipo son, qué valores suelen tomar y cuáles pueden faltar. Con esa descripción del esquema, el sistema es capaz de enfrentarse a bases de datos que no conocía de antemano, que es precisamente lo que le da su capacidad de generalización.

Por último, el ejecutor es el único módulo con acceso a la base de datos. Recibe la consulta ya validada, reconoce de qué tipo es y la lanza contra MongoDB, devolviendo los resultados acotados a un número razonable. Reunir aquí todo el acceso a datos tiene una consecuencia valiosa para la seguridad, ya que las dos garantías más importantes, que el sistema solo lee y nunca modifica, y que jamás devuelve un volumen desmedido de resultados, quedan aseguradas en un solo punto, fácil de revisar.

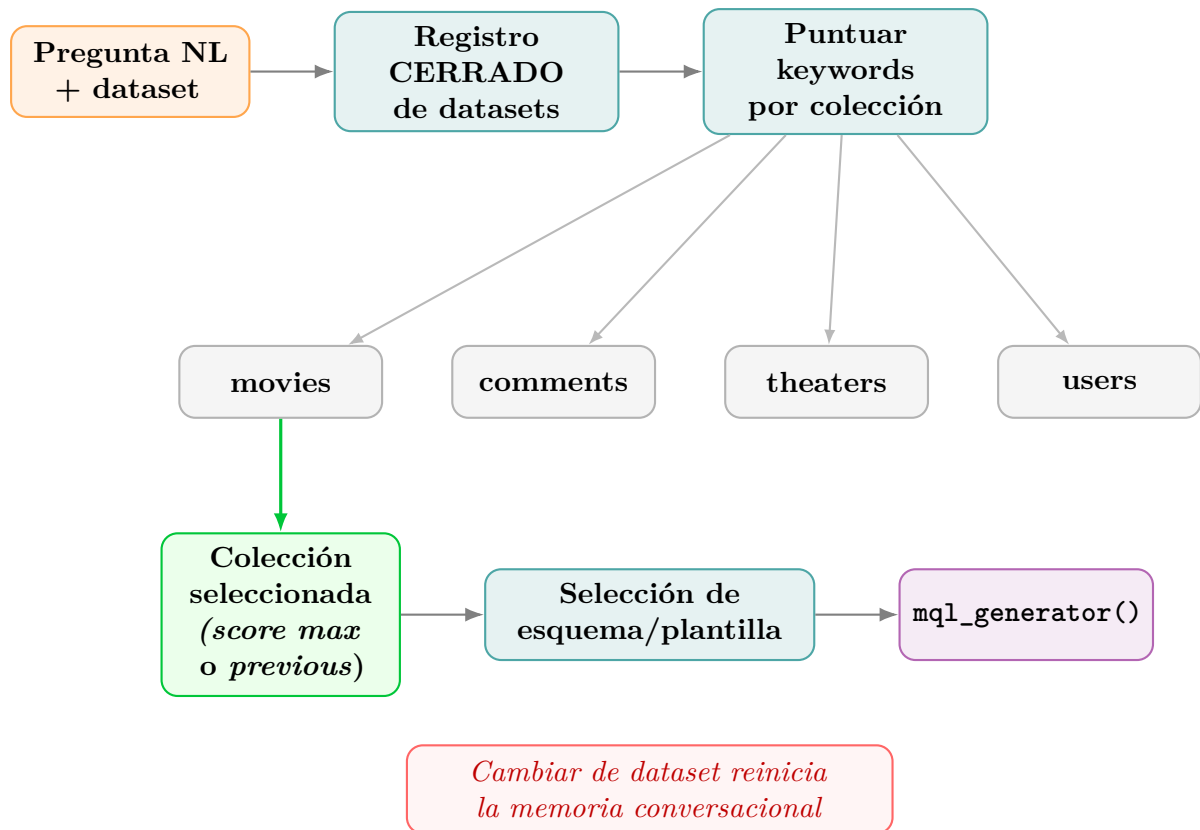


Figura 7: Detección de colección y *routing* multi-dataset en MongoMind. El registro cerrado de datasets actúa como frontera de seguridad, donde `detect_collection()` puntúa la pregunta contra las *keywords* de cada colección y selecciona la de mayor score (o reutiliza la anterior en *follow-ups*).

Cierra el diseño un detalle que atraviesa todo el trayecto de una pregunta, y es que la base de datos sobre la que se trabaja viaja como un dato más a lo largo de la cadena, con un valor por defecto sensato. Gracias a ello, el soporte para varias bases de datos se entreteje en el sistema sin complicar la vida a quien solo trabaja con una. Es un buen ejemplo de cómo una decisión pequeña y discreta puede habilitar una funcionalidad nueva sin romper nada de lo anterior.

5.3. Capa web (FastAPI)

La interacción con el usuario se ofrece a través de una aplicación web construida con FastAPI [25]. Esta se ha consolidado como el framework de referencia en Python para el desarrollo de APIs web modernas, combinando un alto rendimiento con una experiencia de desarrollo basada en anotaciones de tipo y validación automática de datos mediante Pydantic. Esta capa web se limita a orquestar llamadas y a gestionar la sesión.

La aplicación web se organiza en torno a unas pocas rutas que cubren el ciclo completo de interacción. La ruta raíz sirve la interfaz de chat: una página HTML que reúne la caja de pregunta, el desplegable de selección de dataset y el área donde se muestran los resultados.

El núcleo funcional reside en la ruta de consulta, que recibe la pregunta del usuario y, opcionalmente, el dataset sobre el que operar. Su responsabilidad es coordinar el *pipeline* completo. Sanitiza la pregunta, aplica el limitador de frecuencia, recupera el historial de la sesión, invoca la detección de colección y la generación del MQL. Después, ejecuta la consulta resultante, registra la operación en el log de auditoría y compone finalmente la respuesta. Esta respuesta distingue entre los fallos reales (una generación inválida o un error en la ejecución) y los destinados a transmitir información no crítica. El ejemplo más representativo de este segundo caso es el de una consulta que se ejecuta correctamente pero no devuelve ningún resultado, situación que no constituye un error y que, por tanto, no debe presentarse como tal. Existe además una ruta dedicada a reiniciar la memoria conversacional de la sesión, que permite al usuario empezar de cero en cualquier momento.

En todos los casos, el MQL generado se devuelve y se muestra siempre junto a los resultados. Con ello se cumple el requisito de transparencia, ya que tanto el usuario como un eventual revisor pueden auditar con exactitud qué consulta se ha ejecutado.

5.4. Seguridad y robustez

Los requisitos de seguridad se materializan en tres módulos especializados que actúan como barreras independientes alrededor del *pipeline*, más las garantías ya integradas en `mql_generator` (rechazo de escrituras, JSON validados) y en `db_connector` (solo lectura) (Figura 8).

`sanitizer.py` actúa en la entrada. Su función `sanitize_question()` normaliza el Unicode, elimina los caracteres de control, colapsa los espacios redundantes e impone límites de longitud (entre 2 y 500 caracteres). Se invoca antes de que la pregunta llegue a formar parte del *prompt*, de modo que el modelo nunca recibe entradas mal formadas.

`rate_limit.py` protege frente a abusos. Implementa un limitador de ventana deslizante en memoria, sin dependencias externas, que restringe a cada cliente a 20 peticiones por minuto, aquellas peticiones que excedan el límite recibirán un HTTP 429⁷.

`query_logger.py` proporciona la traza de auditoría. Su función `log_query()` añade una línea JSON por cada consulta ejecutada (pregunta, MQL, colección, número de resultados o error, marca temporal y cliente) al fichero `logs/queries.log`. Una propiedad de diseño deliberada es que el registro nunca lanza excepciones, porque un fallo en el propio registro jamás debe interrumpir la atención de la petición del usuario.

5.5. Memoria conversacional

El último elemento del diseño es el que convierte una sucesión de consultas independientes en una conversación. Cada sesión de navegador conserva los últimos cinco intercambios como una lista plana de turnos (`HISTORY_WINDOW` (5)). Este historial se pasa a `mql_generator.generate(..., history=...)`, de modo que el modelo dispone del contexto reciente y puede resolver referencias anafóricas. Preguntas de seguimiento como *¿y ordenadas por año?* o *de esas, solo las posteriores a 2010* se interpretan a la luz de la consulta anterior, no de forma aislada.

⁷Código de estado: *Too Many Requests*

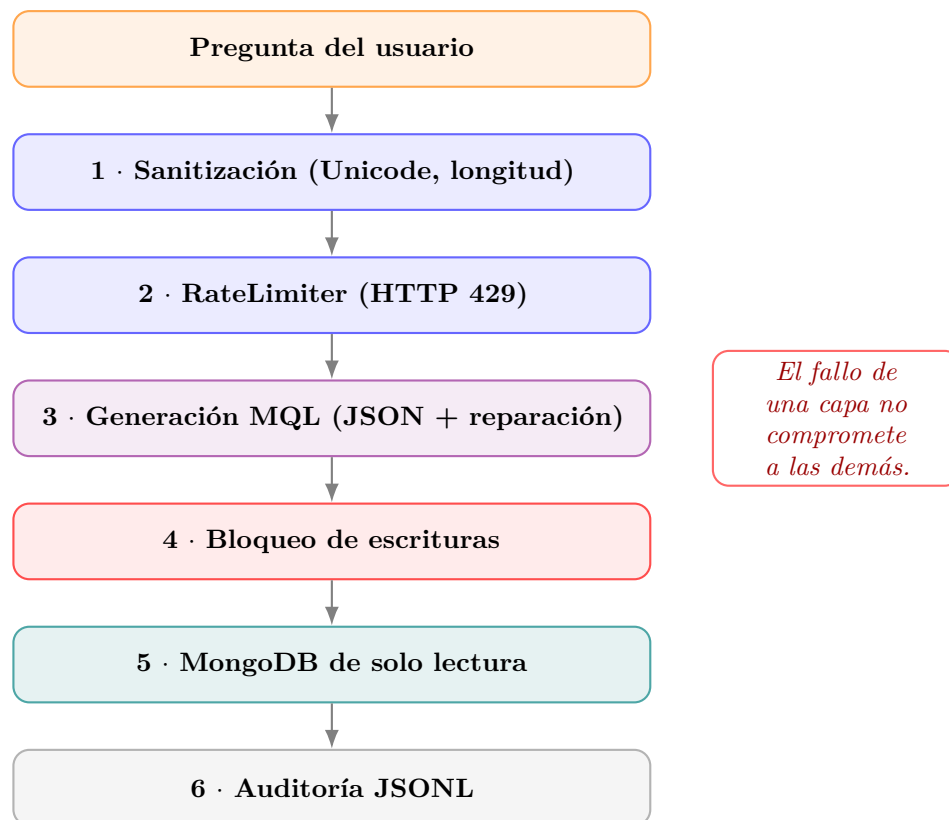


Figura 8: Sanitización de la entrada, limitación de frecuencia, validación del MQL como JSON, rechazo de operadores de escritura, conexión de solo lectura y registro de auditoría.

El diseño toma aquí dos decisiones. La primera es mantener las sesiones en un diccionario en memoria, sin persistencia entre reinicios ya que para un asistente interactivo, la simplicidad y la ausencia de estado en disco pesan más que la durabilidad de un historial efímero. La segunda es centralizar la construcción de los turnos de historial en una función auxiliar en lugar de formatearlos a mano en la capa web. Finalmente, al cambiar de dataset la memoria se vacía, porque una referencia anafórica que cruza de un dataset a otro no tiene sentido.

6. Implementación

Si el capítulo anterior describe la forma del sistema, este describe cómo está construido: se discutirá la tecnología empleada, los algoritmos concretos de las piezas más delicadas y los problemas reales que surgieron al enfrentar el diseño con datos auténticos. Se hace especial hincapié en aquellos puntos donde la implementación tuvo que resolver una dificultad no evidente desde el diseño, porque son los que aportan el conocimiento más transferible del trabajo.

6.1. Stack tecnológico

La implementación se apoya en un conjunto reducido y deliberadamente convencional de tecnologías, todas ellas libres o sin coste de uso (Cuadro 2).

Capa	Tecnología	Papel
Modelo de lenguaje	Ollama + llama3.2 (3B)	Servidor local del LLM; modelo principal de generación de MQL.
Acceso al LLM	SDK <code>ollama</code> de Python	Cliente que invoca al modelo a <code>temperature=0.1</code> .
Capa web	FastAPI	Rutas HTTP, validación de cuerpos y documentación OpenAPI.
Acceso a datos	PyMongo	Conexión y ejecución de consultas contra MongoDB.
Base de datos	MongoDB Atlas	Almacén en la nube de los <i>datasets</i> de ejemplo.
Empaquetado	Docker + Docker Compose	Despliegue reproducible de la aplicación.
Pruebas	pytest	Batería unitaria y de integración.

Cuadro 2: Stack tecnológico utilizado en el proyecto.

El LLM se ejecuta a través de Ollama [20], y se controla mediante el SDK oficial⁸. El modelo concreto se lee de la variable `OLLAMA_MODEL` (por defecto `llama3.2`), lo que permite cambiarlo sin tocar el código. La base de datos, en cambio, no es local sino Atlas: los *datasets* de ejemplo, los esquemas y el *benchmark* residen en la nube, lo que tiene una consecuencia práctica importante para el despliegue. La generación se realiza siempre a temperatura baja (0,1) para acotar la variabilidad de la salida, aun aceptando que un modelo de lenguaje nunca es completamente determinista.

6.2. Generación de MQL

El módulo `mql_generator` concentra la lógica más delicada del sistema, y su implementación se entiende mejor descomponiéndola en las cuatro tareas que encadena: construir el contexto, invocar al modelo, extraer la consulta y validarla (Figura 9).

⁸Un SDK (*Software Development Kit* o Kit de Desarrollo de Software) es un conjunto de herramientas, bibliotecas y recursos empaquetados que nos permite interactuar con una plataforma sin tener que implementar esa comunicación desde cero.

6.2.1. Construcción del contexto y partición del prompt

Cada plantilla de `src/prompts/templates/` es un fichero de texto que contiene el rol del sistema, el esquema completo de la colección, entre ocho y diez pares (pregunta → MQL) verificados, y una línea final de la forma *Ahora responde...* Todo lo anterior se convierte en el mensaje de sistema, y la pregunta del usuario se añade como un turno nuevo. La pregunta real se inyecta como un turno de conversación independiente, no sustituyendo ese marcador. Entre el mensaje de sistema y la pregunta actual se intercala, si existe, el historial reciente de la sesión, de modo que el modelo recibe el contexto completo en el formato de mensajes que espera el SDK.

Cuando la colección no tiene plantilla curada, el contexto se genera dinámicamente a partir del esquema inferido. Para evitar que un único campo desmesurado infle el *prompt*, los valores de ejemplo de la plantilla dinámica se truncan a una longitud máxima (`_MAX_EXAMPLE_LEN`). Tanto los esquemas como las plantillas se cachean con clave (`database`, `collection`), de manera que una misma colección presente en dos datasets distintos no provoque problemas.

6.2.2. Invocación del modelo

La llamada al LLM se realiza mediante el SDK de Ollama con los mensajes contruidos y la temperatura fijada. El nombre del modelo se resuelve en el momento de la llamada, a través de una función interna `_model()` que lee `OLLAMA_MODEL` en cada invocación; este detalle, aparentemente menor, es lo que permite que la opción `-model` de la evaluación cambie el modelo en caliente sin reimportar el módulo.

6.2.3. Extracción de la consulta.

La respuesta del modelo es texto, y rara vez es JSON puro: suele venir envuelta en explicaciones, vallas de código Markdown o texto introductorio. La función `_extract_json` localiza y aísla el bloque JSON de esa respuesta, distinguiendo además entre un objeto (`{...}`, que corresponde a un `find`) y una lista (`[...]`, que corresponde a un *aggregation pipeline*).

6.2.4. Reparación de JSON *casi válido*

Los modelos locales de tamaño moderado a veces generan JSONs estructuralmente defectuosos, pero reconocible. Estos defectos incluyen claves sin comillas (`size: 5`), comas finales sobrantes y descuidos similares. En lugar de descartar esas respuestas, la función `_repair_json` corrige esos defectos de puntuación, y `generate()` reintenta el parseo una vez tras la reparación antes de darse por vencida. Esta tolerancia recupera consultas que de otro modo se perderían.

6.3. Inferencia dinámica de esquema

La inferencia de esquema es la pieza que hace realmente posible la generalización a colecciones no previstas. La idea de partida es que, como en MongoDB no existe un catálogo que consultar, la única forma de conocer la estructura de una colección es mirar los datos directamente. El módulo examina una muestra de entre 50 y 100 documentos y reconstruye su forma observando lo que encuentra.

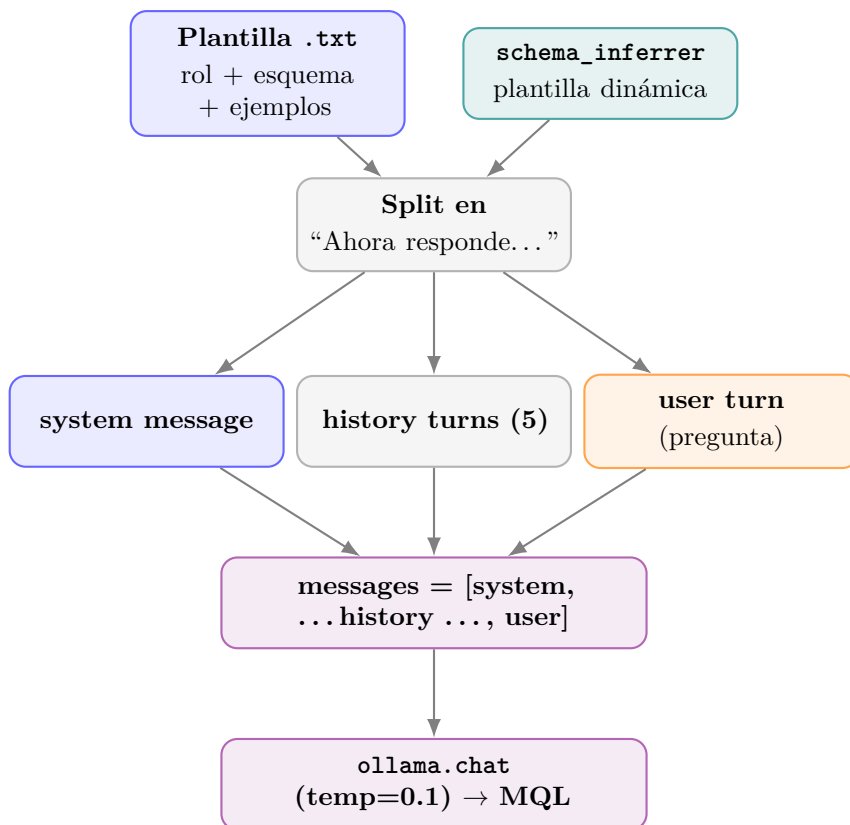


Figura 9: Construcción del prompt few-shot (`_build_messages`).

De esa muestra deduce, para cada campo, cómo se llama y con qué frecuencia aparece, qué tipo o tipos BSON adopta y algunos valores de ejemplo. El análisis no se queda en la superficie, sino que continúa por los subdocumentos anidados y reconoce los *arrays*, anotando de qué tipo son sus elementos. El resultado es una descripción del esquema lo bastante rica para que el modelo entienda con qué campos puede trabajar y de qué tipo son, descripción que después alimenta la plantilla dinámica.

Que esta inferencia funcione es justo lo que permite a MongoMind dar servicio a colecciones que no tienen plantilla curada, como `comments`, `theaters` o `users`, y, sobre todo, a datasets enteros nuevos, apoyándose únicamente en los datos y sin necesidad de reentrenar nada.

6.4. Soporte multi-dataset

El esquema dinámico permite concentrar toda la información en un registro centralizado que actúa como única fuente de verdad. La consecuencia práctica es que dar de alta un dataset nuevo se reduce a añadir una entrada en ese registro, sin tocar nada más.

A partir de ahí, el dataset activo viaja por todo el *pipeline* y la capa web lo expone al usuario mediante un *dropdown menu*, de modo que cambiar de uno a otro es inmediato. Ese cambio reinicia además la memoria conversacional, para evitar que el contexto de un dataset se arrastre al siguiente. Para las colecciones que merecen ejemplos cuidados se han escrito plantillas propias, mientras que todas las demás quedan cubiertas por el esquema inferido; ambos mecanismos conviven sin fricción.

Conviene señalar una consecuencia operativa de la frontera de seguridad. Los datasets de ejemplo distintos del de referencia deben cargarse previamente en Atlas, y esa carga exige un usuario con permisos de escritura. Como el usuario de producción es de solo lectura, no puede realizarla él mismo, lo que recuerda que la decisión de operar siempre en modo lectura tiene un coste real en las tareas de administración.

6.5. Ingeniería de prompts

Dado que toda la adaptación del sistema ocurre vía *prompt*, la calidad de las plantillas es el factor más determinante de la precisión, y su elaboración fue una de las tareas de implementación más importantes. Cada plantilla curada incluye el esquema completo de su colección y entre ocho y diez ejemplos (pregunta → MQL) que se verificaron uno a uno como ejecutables y de resultado no vacío contra Atlas, para no enseñar al modelo patrones que no funcionan.

Más que ofrecer ejemplos, este bloque enuncia de forma directa los principios que el modelo tiende a pasar por alto al construir consultas MQL:

1. **Guarda de tipo en campos numéricos.** Los valores de campos como `imdb.rating`, `runtime` o `year` no siempre son números limpios. Cuando se quiere ordenar por ellos o calcular promedios, conviene asegurarse antes de que el valor es realmente numérico mediante la guarda `{"$type": "number"}`, ya que de lo contrario un dato mal formado puede afectar al resultado.
2. **Despliegue de arrays antes de agrupar.** Cuando se quiere agrupar o contar a partir de los elementos contenidos en un array, primero hay que desplegar ese array con un `$unwind` para tratar cada elemento por separado.
3. **Recuento con \$count con nombre.** El recuento de documentos se expresa con una etapa `$count` a la que se asigna un nombre.
4. **Consultas por número de elementos.** La petición habitual de registros con más de N elementos se modela mejor comprobando si existe la posición `campo.N` que recurriendo a un `$size` mal planteado.

La elección de estas pistas no fue arbitraria, sino que surgió del análisis de los errores que el modelo cometía con más frecuencia. De este modo, la manera de redactar los *prompts* y el estudio de los fallos se retroalimentan formando un ciclo de mejora continua, en el que cada error observado sugiere una nueva indicación para prevenirlo.

6.6. Despliegue con Docker

El empaquetado con Docker persigue la reproducibilidad del despliegue. El contenedor incluye únicamente la aplicación FastAPI, no una base de datos. La razón es que los datos viven en Atlas, de modo que un MongoDB en contenedor estaría vacío y no serviría de nada; toda la información (*datasets*, esquemas y *benchmark*) reside ya en Atlas.

De esa decisión se derivan los dos puntos delicados de la configuración. El contenedor se conecta a Atlas mediante la variable `MONGODB_URI`, leída del fichero `.env`. Como el LLM se ejecuta en Ollama sobre la máquina anfitriona, y no dentro del contenedor, este último necesita alcanzar el servidor del *host*. Para ello basta con apuntar al nombre `host.docker.internal` (que el SDK de Ollama respeta sin modificar el código) y declarar `extra_hosts: [host.docker.internal:host-gateway]` en *compose*, de modo que dicho nombre resuelva correctamente también en Linux. Con esta configuración, el sistema completo se levanta con una sola orden (`docker compose up --build`) y queda accesible en `http://localhost:8000`, siempre que el anfitrión disponga de Docker, tenga Ollama en marcha y cuente con el `.env` que contiene la URI de Atlas (Figura 10).

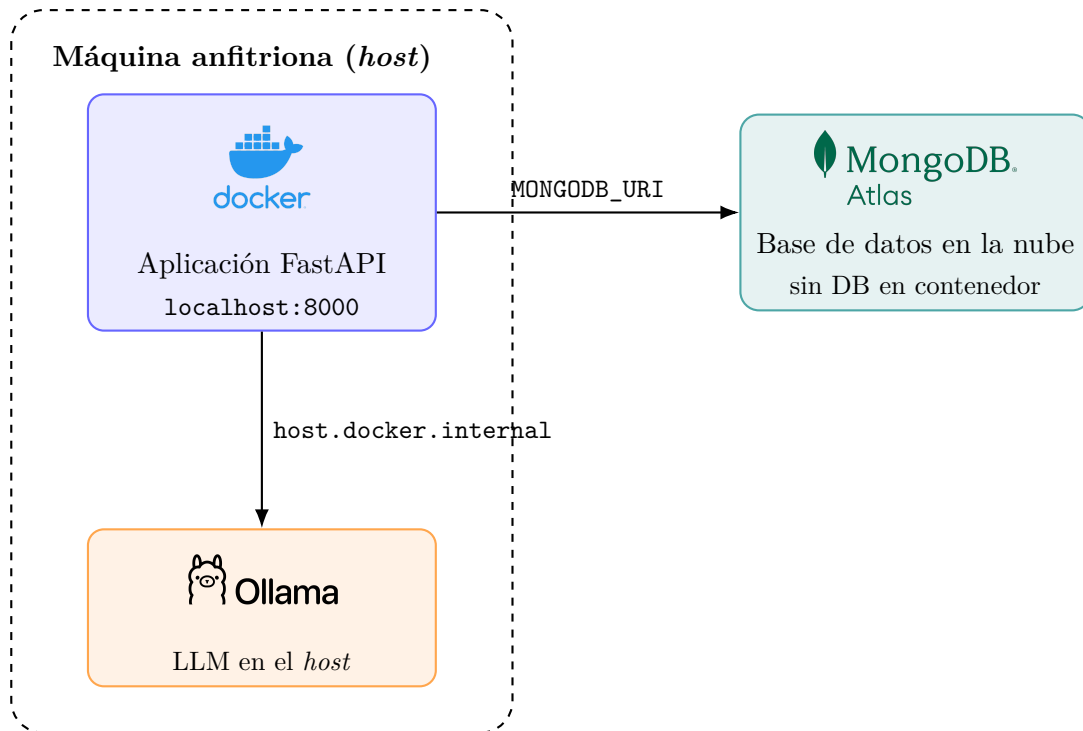


Figura 10: Topología de despliegue de MongoMind. El único componente contenedorizado es la aplicación FastAPI. El servidor Ollama se ejecuta en la máquina anfitriona y la base de datos reside en MongoDB Atlas en la nube (mediante `MONGODB_URI`). De ahí que no exista ninguna base de datos contenedorizada.

7. Resultados

Descritas ya la arquitectura y la implementación, esta sección presenta la evaluación empírica de MongoMind y responde con datos a tres preguntas. La primera, cómo de bien traduce el modelo principal las preguntas a MQL. La segunda, qué tipos de error comete con más frecuencia y cuánto mejora cuando se refinan los ejemplos del *prompt*. Y la tercera, cómo se sitúa frente a otros modelos locales en términos de calidad y de velocidad.

7.1. El conjunto de evaluación

La evaluación se apoya en un *benchmark* propio de 65 pares (NL $\rightarrow$ MQL) sobre la colección *movies* de *sample_mflix*. Cada par se verificó uno a uno como ejecutable y de resultado no vacío contra Atlas, de modo que las consultas de referencia constituyen un suelo fiable con el que comparar.

Las preguntas se reparten en cuatro niveles de complejidad creciente, con un peso mayor en los tramos simple y medio, que son los más frecuentes en un uso real, y una presencia menor pero deliberada de consultas de alta complejidad y de casos ambiguos o con errata (Figura 11). Este último grupo, pequeño pero importante, somete al sistema a preguntas mal formuladas que ponen a prueba su robustez. El banco está prácticamente equilibrado entre inglés y español, con una proporción cercana al cincuenta por ciento en cada lengua, lo que permite comprobar que el sistema no privilegia a ninguna de las dos.

Siguiendo la práctica heredada del estado del arte, el banco se divide en una proporción aproximada de setenta a treinta, reservando el conjunto de prueba como material oculto que nunca se emplea para afinar las plantillas. Esta separación es la que da validez a las cifras finales, ya que mide la capacidad de generalizar y no la de memorizar. La proporción de complejidades se conserva dentro de cada partición, para que ninguna de las dos quede sesgada hacia las preguntas fáciles o las difíciles. El cuarto ángulo describe qué clase de respuesta produce cada consulta, ya que de ello depende la estrategia de comparación que se aplica después. La mayoría devuelve conjuntos de documentos, y un grupo menor produce agrupaciones, recuentos o valores escalares como medias y sumas (Figura 12).

7.2. Rendimiento del modelo principal

Sobre el conjunto de prueba oculto, el modelo principal (11ama3.2, de tres mil millones de parámetros) alcanza una corrección funcional cercana a tres de cada cuatro consultas, y una corrección parcial todavía algo superior cuando se admiten respuestas casi acertadas (Figura 13). El dato más ilustrativo es el contraste entre esa corrección funcional y la coincidencia exacta, mucho más baja. Esa diferencia confirma en la práctica lo argumentado en el estado del arte, ya que muchas consultas generadas son funcionalmente correctas aunque difieran del texto de referencia en el orden de las condiciones o en el nombre de un campo proyectado.

Al desglosar el acierto por dificultad se obtiene una imagen informativa (Figura 14). El sistema resuelve con soltura las preguntas simples y medias, ambas por encima de la media global, y es justo en estos tramos, los más habituales en un uso cotidiano, donde rinde mejor. El acierto desciende en las consultas de alta complejidad, que son las que

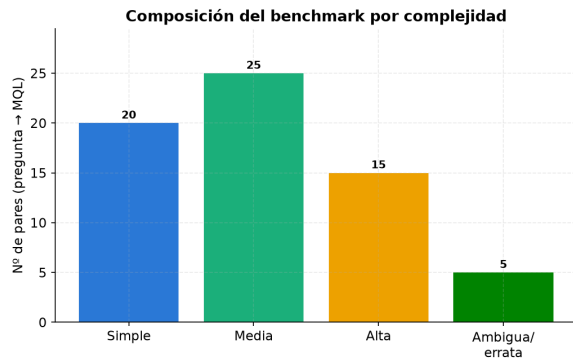


Figura 11: El grueso de las preguntas se concentra en los niveles simple y medio, con una cuota menor de consultas de alta complejidad y de casos ambiguos o con errata.

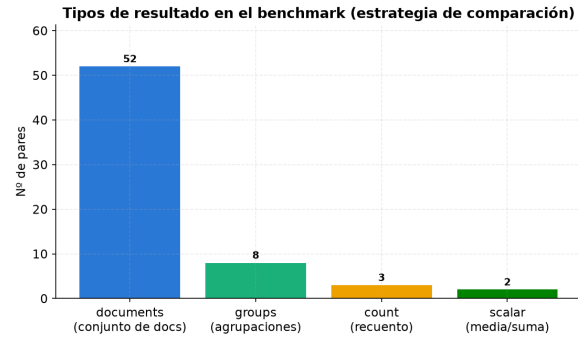


Figura 12: El predominio de los conjuntos de documentos convive con agrupaciones, recuentos y valores escalares, cada uno con su propia estrategia de comparación funcional.

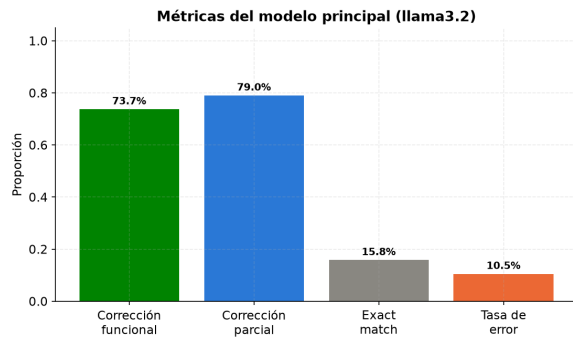


Figura 13: La corrección funcional supera con holgura a la coincidencia exacta, lo que ilustra por qué esta última no es una medida fiel de la utilidad real.

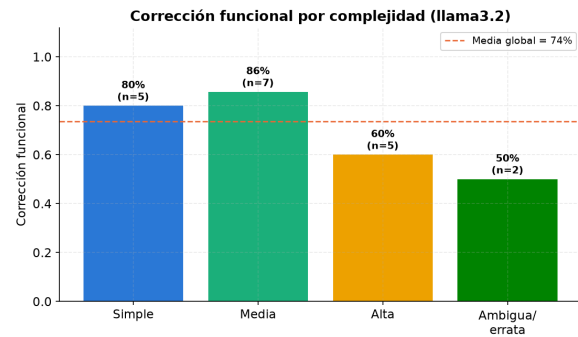


Figura 14: El acierto se mantiene alto en las preguntas simples y medias y cae en las de alta complejidad, donde entran en juego las agregaciones más elaboradas.

exigen encadenar varias etapas de agregación en el orden correcto, desplegar *arrays* antes de agrupar y aplicar guardas de tipo, si bien se mantiene en un nivel apreciable. Las preguntas ambiguas o con errata son las que más se resisten, algo esperable cuando el enunciado de partida es deficiente, aunque el sistema todavía acierta la mitad de ellas, lo que evidencia una robustez razonable frente a entradas imperfectas. El margen de mejora del sistema está, por tanto, en el razonamiento sobre el *aggregation pipeline*.

7.3. Análisis de errores y refinamiento de los ejemplos

Para entender en qué falla el modelo se clasificaron, sobre el conjunto de desarrollo, todos los errores cometidos en una pequeña taxonomía (Figura 15). El patrón resultó muy concentrado. Casi la mitad de los fallos respondían a una sola causa, la ausencia de la guarda de tipo `{"$type": "number"}` al ordenar o promediar campos como la puntuación o el año, que en *sample_mflix* contienen a veces la cadena vacía. El resto de errores aparecía de forma aislada, como recuentos mal formados o filtros innecesariamente complicados.

A partir de ese diagnóstico se reforzaron las plantillas con un bloque de reglas explícitas y un puñado de ejemplos verificados, orientados precisamente a los fallos dominantes.

El efecto fue notable (Figura 16). En el conjunto de desarrollo la corrección funcional pasó del 73,9 % al 89,1 %, una ganancia de más de quince puntos. Lo decisivo es que esa mejora se trasladó también al conjunto de prueba oculto, que ni se tocó ni se usó para afinar nada, donde el acierto subió alrededor de diez puntos. Que la ganancia generalice al material oculto indica que no hubo sobreajuste a los ejemplos vistos, sino aprendizaje de patrones útiles. Todo ello se consiguió sin modificar un solo parámetro del modelo, actuando únicamente sobre su contexto, lo que valida en la práctica la apuesta del proyecto por la adaptación vía *prompt* frente al reentrenamiento.

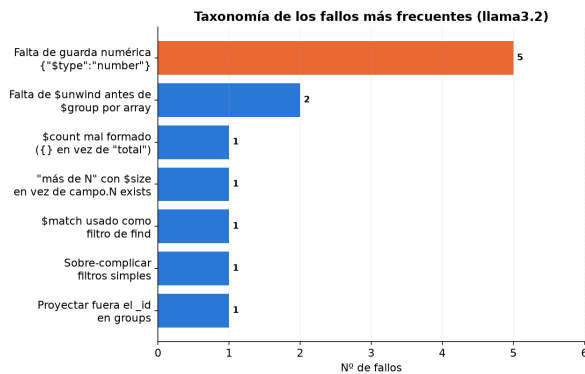


Figura 15: Una única causa, la falta de guarda numérica, concentra buena parte de los errores, seguida de la omisión del despliegue de *arrays*.

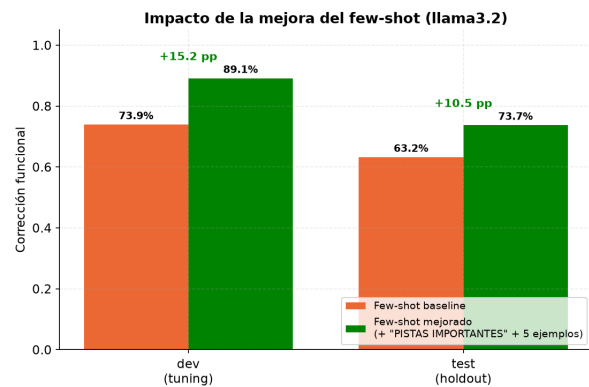


Figura 16: La mejora se observa tanto en desarrollo como en el conjunto de prueba oculto.

7.4. Comparativa entre modelos locales

La última pregunta es si la elección del modelo principal estaba justificada. Para responderla se evaluó sobre el mismo conjunto de prueba un grupo de modelos locales de tamaños distintos (*llama3.1* de ocho mil millones de parámetros [6], *qwen2.5:3b* de tres mil millones [23], y *gemma2:2b* de dos mil millones [27]), junto con *nl2mongo* un modelo pequeño especializado en la tarea NL→MQL pero entrenado sobre esquemas ajenos a los de este trabajo [4]. El ranking por corrección funcional ordena el panorama (Figura 17). A la cabeza se sitúan el modelo principal (*llama3.2*) y *llama3.1*, separados por un margen menor que la propia variación entre ejecuciones, de modo que cabe considerarlos equivalentes en calidad. Tras ellos, *qwen2.5:3b* se queda algo por debajo, *gemma2:2b* descendiendo un escalón más y *nl2mongo* se desploma. Este último resultado es en sí mismo instructivo, ya que un modelo afinado sobre esquemas y convenciones ajenos a *sample_mflix* apenas acierta, lo que ilustra el problema de portabilidad de las soluciones especializadas y refuerza el valor de un enfoque adaptable por contexto.

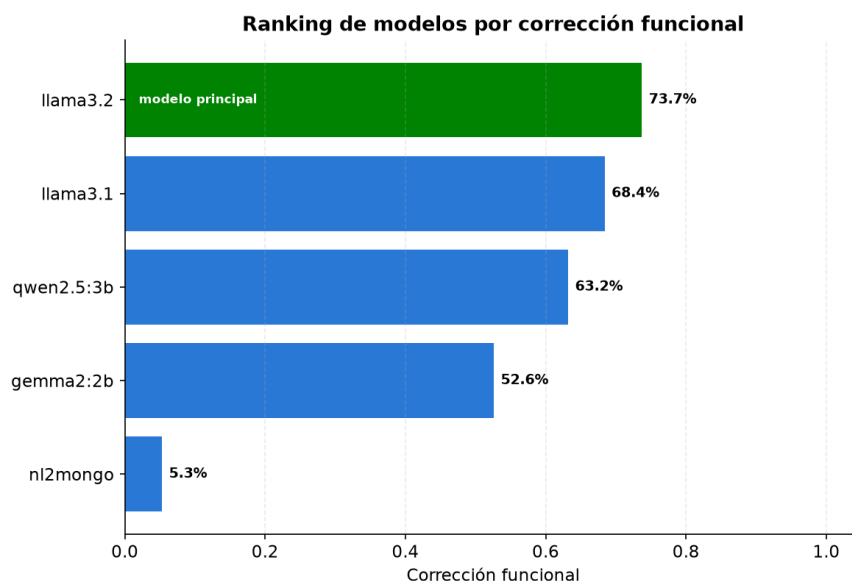


Figura 17: El modelo principal encabeza la tabla, empatado dentro del margen de variación con el mayor de todos pese a tener una fracción de sus parámetros.

Esa fotografía global se enriquece al examinar las métricas en detalle (Figura 18). llama3.1 destaca sobre todo en coincidencia exacta, muy por encima del resto, lo que indica que tiende a reproducir con más fidelidad la forma textual de las consultas de referencia, aunque esa fidelidad no se traduzca en una corrección funcional mayor. En el extremo opuesto, gemma2:2b exhibe la tasa de error más alta, prueba de que un modelo tan pequeño tiene dificultades para emitir siquiera un JSON bien formado de manera consistente. Al cruzar este comportamiento con la complejidad de las preguntas se aprecia que las diferencias entre modelos se ensanchan en los tramos más exigentes, mientras que en las consultas simples casi todos rinden de forma parecida (Figura 19).

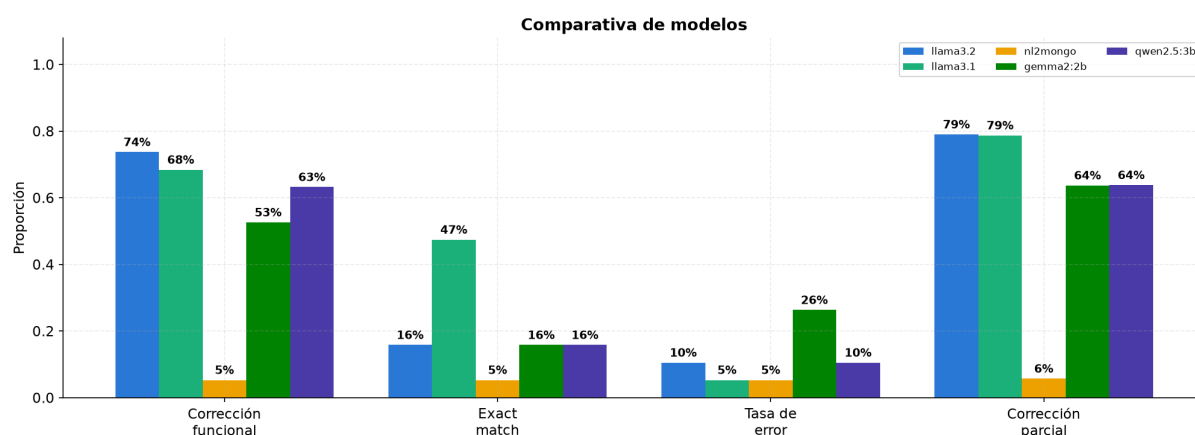


Figura 18: El modelo mayor destaca en coincidencia exacta y el generalista más pequeño acumula la mayor tasa de error.

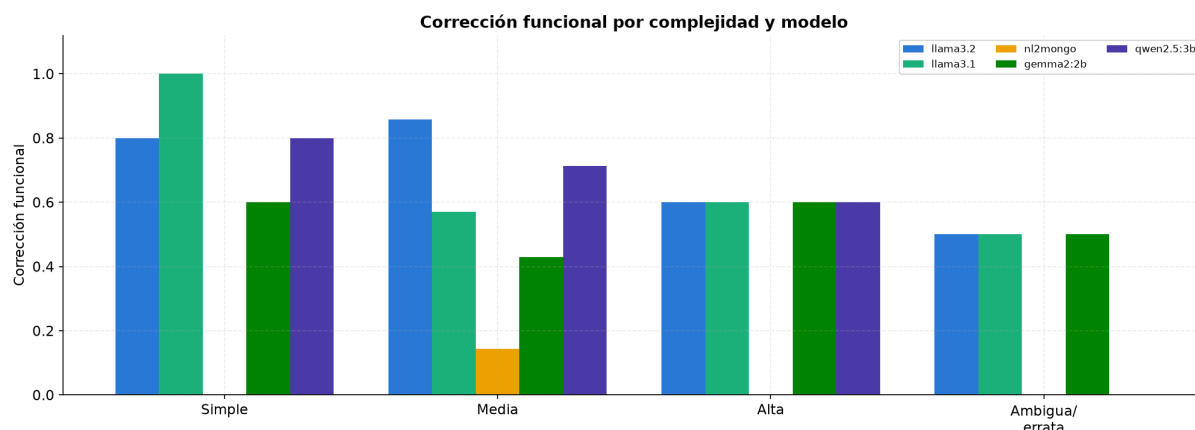


Figura 19: La precisión disminuye conforme crece la complejidad de la consulta (más operadores y etapas).

La métrica que cierra la decisión es el compromiso entre calidad y velocidad (Figura 20). Situando cada modelo según su acierto y su latencia media, el modelo principal queda en la esquina más favorable de todas, la del mayor acierto con el menor tiempo de generación a la vez. Los modelos de mayor tamaño, como el de ocho mil millones de parámetros, alcanzan una calidad del mismo orden, pero a costa de una latencia varias veces superior, casi el cuádruple en ese caso. Dado que las diferencias de acierto entre los mejores modelos caben dentro de la variación entre ejecuciones, el factor que de verdad los separa es la velocidad. Es justamente este equilibrio, y no la máxima precisión a cualquier coste, el que aconseja adoptarlo como modelo principal en un sistema interactivo donde la fluidez de la respuesta forma parte de la experiencia. Conviene recordar, por último, que la salida de un modelo de lenguaje conserva cierta variabilidad incluso a temperatura baja, por lo que las cifras concretas pueden oscilar ligeramente entre ejecuciones sin alterar las conclusiones generales.

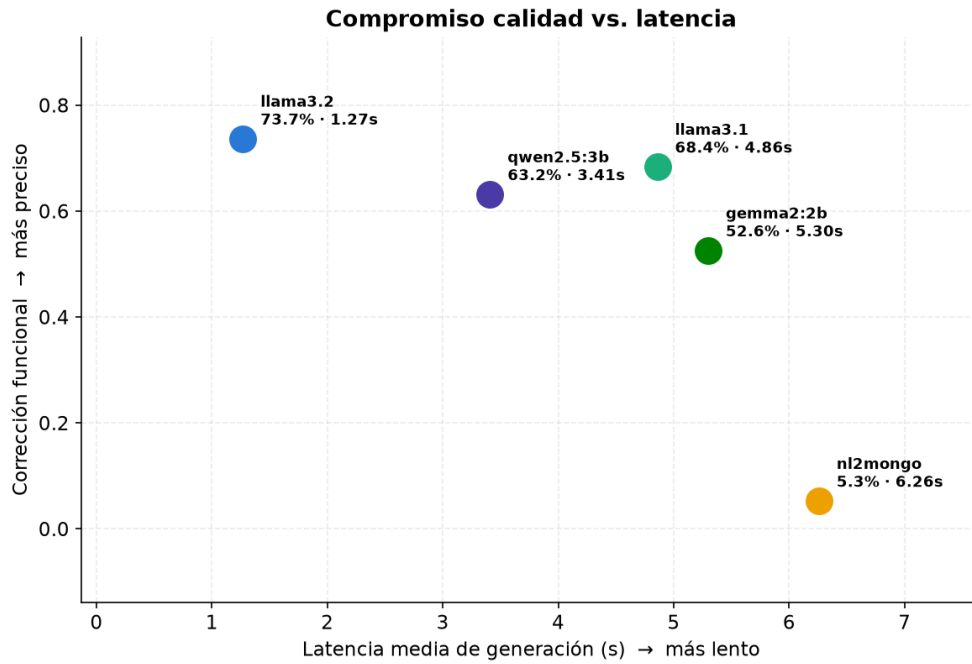


Figura 20: El modelo principal se ubica en la región más conveniente, con una corrección comparable a la de modelos mayores pero a una fracción de su tiempo de generación.

8. Conclusiones

8.1. Limitaciones

La primera limitación nace del propio instrumento de medida. El *benchmark* consta de 65 pares, de los cuales 19 forman el conjunto de prueba, lo que significa que cada pregunta pesa más de cinco puntos porcentuales sobre la cifra final. En consecuencia, la corrección funcional oscila de forma apreciable entre ejecuciones aunque no cambie nada del sistema, una variabilidad agravada por la naturaleza estocástica de los modelos de lenguaje, que no producen una salida idéntica ni siquiera a temperatura baja. Las cifras presentadas deben leerse, por tanto, como una fotografía representativa y no como un valor exacto y repetible.

Una segunda limitación afecta al alcance del dominio. El sistema opera sobre un registro cerrado de tres conjuntos de datos, una decisión que aporta seguridad y fiabilidad pero que, al mismo tiempo, lo aleja de un conector universal capaz de enfrentarse a cualquier base de datos MongoDB. Además, la evaluación cuantitativa se concentra en la colección **movies**, mientras que el resto de colecciones y de datasets se ejercita de forma más cualitativa. La capacidad de generalización queda así demostrada de manera convincente, pero no medida con el mismo rigor en todo el dominio.

La detección de la colección constituye un tercer punto débil. Al apoyarse en la coincidencia de palabras clave y no en un modelo de aprendizaje, el enrutado resulta transparente y predecible, pero también frágil ante preguntas que no contienen ningún término revelador o que mezclan vocabulario de varias colecciones. En esos casos el sistema puede dirigir la consulta a una colección equivocada, con el consiguiente error.

Por último, el rendimiento decae en las consultas de mayor complejidad, justamente las que exigen encadenar varias etapas de agregación, desplegar *arrays* y razonar sobre el orden de las operaciones. Un modelo local de tamaño moderado tiene aquí su techo natural, y la decisión deliberada de no recurrir al reentrenamiento, acertada por sus ventajas de portabilidad y de coste, fija un límite superior de precisión que solo puede mejorarse actuando sobre el contexto del *prompt*.

8.2. Trabajo futuro

La continuación más inmediata pasa por reforzar la evaluación. Ampliar el *benchmark* con más pares, más colecciones y un reparto de idiomas todavía mayor reduciría la varianza de las cifras, y promediar varias ejecuciones por modelo en lugar de tomar una sola corrida ofrecería una medida más estable y defendible. En un plano más ambicioso, el hueco detectado en el estado del arte invita a contribuir a la construcción de un *benchmark* público y compartido para la tarea, del que hoy carece la comunidad y que permitiría comparar trabajos entre sí.

En la generación de la consulta hay margen para incorporar técnicas de *prompting* más sofisticadas. El razonamiento encadenado [31], que pide al modelo explicitar los pasos antes de responder, y la descomposición de la consulta en subproblemas [22] han demostrado su valor en la tarea homóloga sobre SQL y encajarían bien con el carácter procedimental del *aggregation pipeline*, que es donde el sistema más sufre. Especialmente prometedor resulta un bucle de autocorrección, en el que un error de ejecución o un resultado vacío se devuelven al modelo para que revise y reformule su propia consulta antes de darla por buena.

El enrutado de colecciones podría evolucionar de la coincidencia de palabras clave a un clasificador ligero basado en *embeddings*, más robusto frente a preguntas sin términos evidentes y sin renunciar por ello a la transparencia. En cuanto a los modelos, conviene seguir explorando alternativas locales que mejoren la alta complejidad sin disparar la latencia, y valorar un ajuste fino ligero sobre un corpus propio, cuando se disponga de él, comparándolo de forma justa con el enfoque actual basado únicamente en el contexto.

En el plano del producto quedan abiertas varias mejoras de experiencia. Redactar la respuesta final en lenguaje natural a partir de los documentos recuperados acercaría el sistema a una conversación genuina. La persistencia de la memoria entre sesiones, la autenticación de usuarios y un estudio de usabilidad con participantes reales completarían la madurez del asistente.

8.3. Conclusión final

MongoMind cumple el objetivo para el que fue concebido: permitir a una persona consultar una base de datos MongoDB con preguntas en lenguaje natural, en español o en inglés, sin necesidad de conocer MQL. El sistema traduce la pregunta, valida la consulta, la ejecuta contra datos reales y devuelve el resultado acompañado de la propia consulta generada, de modo que el usuario o un revisor puedan auditar en todo momento qué se ha ejecutado.

El trabajo confirma la viabilidad de su apuesta central, que es resolver una tarea compleja de generación de código apoyándose por completo en la adaptación vía *prompt*, con un modelo de lenguaje local como único componente que aprende y sin reentrenamiento alguno. Esta elección dota al sistema de portabilidad, coste nulo de API y privacidad, y le permite acoger nuevas colecciones y datasets con solo describir su esquema y unos pocos ejemplos. La mejora obtenida al refinar esos ejemplos, sin alterar un solo parámetro del modelo, muestra el alcance de este planteamiento.

En el plano de los resultados, un modelo pequeño y veloz alcanza una corrección funcional del 73,7 % sobre el conjunto de prueba oculto, con una latencia de poco más de un segundo, y compite de igual a igual con alternativas mayores y mucho más lentas. La evaluación se ha realizado, además, con la métrica más fiel a la utilidad real, comparando los resultados de ejecución y no el texto de las consultas, siguiendo el consenso del estado del arte.

La aportación de fondo del proyecto es haber llevado este enfoque al terreno documental, mucho menos explorado que el relacional. Las tres dificultades propias de MongoDB que se identificaron en el estado del arte, la inferencia del esquema, la heterogeneidad de tipos y el carácter procedimental del *aggregation pipeline*, se tradujeron en decisiones de diseño concretas que constituyen el conocimiento más transferible del trabajo. Todo ello sin renunciar a la seguridad, que vertebra el sistema de principio a fin mediante barreras independientes que validan la salida del modelo antes de que llegue a actuar sobre datos reales.

MongoMind demuestra que es posible construir un asistente conversacional fiable, seguro y respetuoso con la privacidad sobre una base de datos documental, con recursos modestos y sin renunciar al rigor en la evaluación. El trabajo deja un camino abierto en un problema tan útil como poco transitado.



9. Referencias

- [1] Ion Androutsopoulos, Graeme D Ritchie, and Peter Thanisch. Natural language interfaces to databases—an introduction. *Natural language engineering*, 1(1):29–81, 1995.
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [4] Chirayu Tripathi. <https://huggingface.co/Chirayu/nl2mongo>, 2026.
- [5] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145, 2024.
- [6] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [7] Murilo Gustineli. The art of tokenization: Breaking down text for ai | towards data science, Sept 2024.
- [8] Gary G. Hendrix. . *Natural-Language Interface*.
- [9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [10] Fei Li and Hosagrahar V Jagadish. Constructing an interactive natural language interface for relational databases. *Proc. VLDB Endow.*, 8(1):73–84, 2014.
- [11] Jieyu Li, Zhi Chen, Lu Chen, Zichen Zhu, Hanqi Li, Ruisheng Cao, and Kai Yu. Dir: A large-scale dialogue rewrite dataset for cross-domain conversational text-to-sql. *Applied Sciences*, 13(4):2262, Feb 2023.
- [12] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, 36:42330–42357, 2023.
- [13] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.
- [14] Jinwei Lu, Yuanfeng Song, Zhiqian Qin, Haodi Zhang, Chen Zhang, and Raymond Chi-Wing Wong. Bridging the gap: Enabling natural language queries for nosql databases through text-to-nosql translation. *arXiv preprint arXiv:2502.11201*, 2025.

- [15] MongoDB, Inc. Available sample datasets for atlas clusters. <https://www.mongodb.com/docs/atlas/sample-data/>, 2024. Consultado en 2026.
- [16] MongoDB, Inc. Mongodb manual. <https://www.mongodb.com/docs/manual/>, 2024. Consultado en 2026.
- [17] MongoDB, Inc. Sample airbnb listings dataset. <https://www.mongodb.com/docs/atlas/sample-data/sample-airbnb/>, 2026. Documentación oficial de MongoDB Atlas. Consultado el 1 de julio de 2026.
- [18] MongoDB, Inc. Sample analytics dataset. <https://www.mongodb.com/docs/atlas/sample-data/sample-analytics/>, 2026. Documentación oficial de MongoDB Atlas. Consultado el 1 de julio de 2026.
- [19] MongoDB, Inc. Sample mflix dataset. <https://www.mongodb.com/docs/atlas/sample-data/sample-mflix/>, 2026. Documentación oficial de MongoDB Atlas. Consultado el 1 de julio de 2026.
- [20] Ollama. Ollama. <https://ollama.com/>, 2024.
- [21] Ana-Maria Popescu, Oren Etzioni, and Henry Kautz. Towards a theory of natural language interfaces to databases. In *Proceedings of the 8th international conference on Intelligent user interfaces*, pages 149–157, 2003.
- [22] Mohammadreza Pourreza and Davood Rafiei. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in neural information processing systems*, 36:36339–36348, 2023.
- [23] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [24] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. Evaluating the text-to-SQL capabilities of large language models. *arXiv preprint arXiv:2204.00498*, 2022.
- [25] Sebastián Ramírez. FastAPI. <https://fastapi.tiangolo.com/>.
- [26] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pages 9895–9901, 2021.
- [27] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [28] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [30] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 7567–7578, 2020.
- [31] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [32] William A Woods. Lunar rocks in natural english: Explorations in natural language question answering. 1977.
- [33] Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, et al. Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 1962–1979, 2019.
- [34] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 3911–3921, 2018.
- [35] Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, et al. Sparc: Cross-domain semantic parsing in context. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4511–4523, 2019.
- [36] Yifei Zhao, Zhaohui Li, and Siyi Lv. Enhancing ai system privacy: An automatic tool for achieving gdpr compliance in nosql databases. *Computers, Materials Continua*, 80(1):217–234, 2024.
- [37] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2SQL: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*, 2017.

10. Anexos

10.1. Selección del *benchmark* de evaluación.

El *benchmark* de evaluación está formado en total por sesenta y cinco pares de pregunta y consulta MQL de referencia sobre la colección `movies`. Abarca los cuatro niveles de complejidad y los dos idiomas, e indica para cada par su nivel, su idioma y el tipo de resultado esperado, que puede ser documentos, recuento, escalar o grupos. Todas las consultas de referencia están verificadas como ejecutables y no vacías contra la base de datos real. El Cuadro 3 resume la distribución del conjunto completo por nivel e idioma, y los Cuadros 4 a 7 reproducen una muestra representativa de cada nivel. El nivel ambiguo conserva las erratas originales de las preguntas, como *dirijio* o *horroror*, para evaluar la robustez del sistema ante entradas imperfectas. El conjunto completo, con sus sesenta y cinco pares y la partición entre desarrollo y prueba, está disponible en el repositorio del proyecto en el fichero `data/benchmark/movies_benchmark.json`.

10.2. Plantilla *few-shot* completa de la colección `movies`.

La plantilla *few-shot* completa de la colección `movies` se ensambla a partir de cinco bloques: el rol de sistema, las reglas estrictas, las pistas para evitar errores comunes, el esquema íntegro de la colección y los ejemplos verificados de pregunta y consulta. El Cuadro 8 recoge las reglas y pistas, el Cuadro 9 el esquema de campos y el Cuadro 10 los ejemplos.

10.3. Ejemplo de esquema inferido dinámicamente.

El módulo de inferencia muestrea alrededor de cien documentos y deduce los campos, sus tipos y un valor de ejemplo. El Cuadro 11 presenta la salida real del inferidor sobre la colección `comments`, que carece de plantilla curada y sostiene por sí sola la capacidad de generalización del sistema. Sobre esta estructura se levanta después una plantilla dinámica equivalente en espíritu a la del Cuadro 9.

10.4. Trazas de interacción.

Se incluyen tres trazas completas de interacción que muestran la pregunta, el MQL generado y el desenlace. La primera ilustra una agregación con guarda numérica; la segunda, una conversación de dos turnos con una referencia anafórica resuelta sobre el turno anterior; la tercera, el rechazo de una petición de escritura por la capa de seguridad. El Cuadro 12 resume las tres.

10.5. Comparativa numérica de modelos.

El Cuadro 13 recoge las cifras numéricas de la comparativa de modelos sobre el conjunto de prueba, que en las figuras del capítulo de resultados aparecen solo de forma gráfica. Todos los modelos comparten el mismo *prompt* y el mismo *benchmark*, y se evalúan por corrección funcional. El modelo principal, `llama3.2`, encabeza la corrección funcional con la menor latencia, empatado dentro del margen de variación con `llama3.1`, que destaca en coincidencia exacta. Las columnas de corrección y error se expresan en porcentaje y la de latencia en segundos por consulta.

10.6. Interfaz web del asistente.

La Figura 21 muestra la interfaz web de chat de MongoMind, el producto final con el que interactúa la persona usuaria. La consulta MQL generada se presenta siempre junto al resultado, de modo que cualquier respuesta pueda auditarse en el momento.

Nivel	Español	Inglés	Total
Simple	10	10	20
Medio	12	13	25
Alto	8	7	15
Ambiguo o errata	2	3	5
Total	32	33	65

Cuadro 3: Distribución de los sesenta y cinco pares del *benchmark* por nivel de complejidad e idioma.

ID	Idioma	Pregunta	MQL de referencia
s01	inglés	Find the movie titled Inception	<code>{"title": "Inception"}</code>
s02	español	Películas del género Action	<code>{"genres": "Action"}</code>
s10	inglés	Comedy movies released in 2010	<code>{"genres": "Comedy", "year": 2010}</code>
s16	español	Películas clasificadas R del género Crime	<code>{"rated": "R", "genres": "Crime"}</code>
s19	español	Películas dirigidas por Quentin Tarantino	<code>{"directors": "Quentin Tarantino"}</code>

Cuadro 4: Muestra del nivel **simple**. Resultado esperado: documentos.

ID	Idioma	Tipo	Pregunta	MQL de referencia
m01	inglés	docs.	Movies with an IMDb rating above 8.5	<code>{"imdb.rating": {"\$gt": 8.5}}</code>
m04	español	docs.	Películas estrenadas entre 2000 y 2010	<code>{"year": {"\$gte": 2000, "\$lte": 2010}}</code>
m06	inglés	docs.	Movies directed by Christopher Nolan or Steven Spielberg	<code>{"directors": {"\$in": ["Christopher Nolan", "Steven Spielberg"]}}</code>
m10	español	docs.	Las 10 películas con mayor puntuación IMDb	<code>[{"\$match": {"imdb.rating": {"\$type": "number"}}}, {"\$sort": {"imdb.rating": -1}}, {"\$limit": 10}, {"\$project": {"title": 1, "imdb.rating": 1, "year": 1, "_id": 0}}]</code>
m16	español	recuento	¿Cuántas películas hay de género Comedy?	<code>[{"\$match": {"genres": "Comedy"}}, {"\$count": "total"}]</code>
m18	inglés	escalar	What is the average runtime of the movies?	<code>[{"\$match": {"runtime": {"\$type": "number"}}}, {"\$group": {"_id": null, "duracion_media": {"\$avg": "\$runtime"}}}]</code>

Cuadro 5: Muestra del nivel **medio**. La columna *Tipo* indica el resultado esperado.

ID	Idioma	Tipo	Pregunta	MQL de referencia
a01	inglés	grupos	How many movies are there of each genre?	[{"\$unwind": "\$genres"}, {"\$group": {"_id": "\$genres", "total": {"\$sum": 1}}}, {"\$sort": {"total": -1}}]
a03	inglés	grupos	Average IMDb rating by genre	[{"\$match": {"imdb.rating": {"\$type": "number"}}}, {"\$unwind": "\$genres"}, {"\$group": {"_id": "\$genres", "rating_medio": {"\$avg": {"\$imdb.rating"}}}, {"\$sort": {"rating_medio": -1}}]
a05	inglés	docs.	How many comments does the movie The Dark Knight have?	[{"\$match": {"title": "The Dark Knight"}}, {"\$lookup": {"from": "comments", "localField": "_id", "foreignField": "movie_id", "as": "comentarios"}}, {"\$project": {"title": 1, "num_comentarios": {"\$size": "\$comentarios"}, "_id": 0}}]
a08	español	grupos	Duración media de las películas por clasificación	[{"\$match": {"runtime": {"\$type": "number"}}}, {"\$group": {"_id": "\$rated", "duracion_medio": {"\$avg": {"\$runtime"}}}, {"\$sort": {"duracion_medio": -1}}]
a11	español	docs.	Películas con un reparto de más de 3 actores	{"cast.3": {"\$exists": true}}
a13	español	escalar	Número total de premios ganados por las películas de Christopher Nolan	[{"\$match": {"directors": "Christopher Nolan"}}, {"\$group": {"_id": null, "total_premios": {"\$sum": "\$awards.wins"}}]

Cuadro 6: Muestra del nivel **alto**, con etapas de *unwind*, *lookup* y agregación.

ID	Idioma	Tipo	Pregunta	MQL de referencia
x02	español	recuento	cuantas películas dirigió Tarantino?	<code>[{"\$match": {"directors": "Quentin Tarantino"}}, {"\$count": "total"}]</code>
x03	español	docs.	las mejores películas	<code>[{"\$match": {"imdb.rating": {"\$type": "number"}}}, {"\$sort": {"imdb.rating": -1}}, {"\$limit": 10}, {"\$project": {"title": 1, "imdb.rating": 1, "_id": 0}}]</code>
x04	inglés	docs.	recent horror movies	<code>{"genres": "Horror", "year": {"\$gte": 2010}}</code>

Cuadro 7: Muestra del nivel **ambiguo o con errata**. Las preguntas conservan las erratas originales para evaluar la robustez del sistema.

Tipo	Instrucción
Regla	Responder únicamente con el bloque JSON de la query, sin explicaciones ni texto adicional.
Regla	El JSON debe ser parseable directamente, sin comentarios en su interior.
Regla	Objeto JSON para consultas simples (<i>filter</i> de find); array JSON para agrupación, ordenación, <i>lookup</i> o etapas múltiples (<i>pipeline</i> de aggregate).
Regla	Nunca usar operadores de escritura: insertOne , updateMany , deleteMany , drop , \$out , \$merge .
Regla	Ante una pregunta ambigua, generar la query más razonable posible.
Pista	Antes de ordenar, promediar o agrupar por imdb.rating , runtime o year , filtrar los numéricos con <code>{"\$type": "number"}</code> : algunos documentos tienen texto vacío en esos campos y romperían el orden o la media.
Pista	Para agrupar o promediar por un campo array (genres , cast , directors), aplicar antes <code>{"\$unwind": "\$campo"}</code> .
Pista	Para "más de N elementos" en un array, usar <code>{"campo.N": {"\$exists": true}}</code> ; por ejemplo, más de 3 actores se expresa como <code>{"cast.3": {"\$exists": true}}</code> , nunca como <code>{"\$size": {"\$gt": N}}</code> .
Pista	\$count lleva siempre un nombre de campo string: <code>{"\$count": "total"}</code> , nunca <code>{"\$count": {}}</code> .
Pista	Para una consulta simple, usar un objeto <code>{filtro}</code> directo; \$match solo va dentro de un <i>pipeline</i> (array).
Pista	No proyectar fuera el _id en agrupaciones, ya que identifica cada grupo.

Cuadro 8: Reglas estrictas y pistas de la plantilla *few-shot* de movies.

Campo	Tipo	Descripción y ejemplo
<code>title</code>	string	Título de la película. Ej.: <i>The Dark Knight</i>
<code>plot</code>	string	Sinopsis corta.
<code>genres</code>	array<string>	Géneros: Action, Adventure, Animation, Biography, Comedy, Crime, Documentary, Drama, Family, Fantasy, History, Horror, Music, Musical, Mystery, Romance, Sci-Fi, Short, Sport, Thriller, War, Western.
<code>cast</code>	array<string>	Actores. Ej.: [<i>Christian Bale, Heath Ledger</i>]
<code>directors</code>	array<string>	Directores. Ej.: [<i>Christopher Nolan</i>]
<code>countries</code>	array<string>	Países de producción. Ej.: [<i>USA</i>]
<code>languages</code>	array<string>	Idiomas. Ej.: [<i>English</i>]
<code>year</code>	integer	Año de estreno. Ej.: 2008
<code>runtime</code>	integer	Duración en minutos. Ej.: 152
<code>rated</code>	string	Clasificación: G, PG, PG-13, R, TV-14, TV-MA, TV-PG, Not Rated.
<code>type</code>	string	"movie" o "series".
<code>released</code>	date	Fecha de estreno. Ej.: <code>ISODate("2008-07-18T00:00:00Z")</code>
<code>num_mflix_comments</code>	integer	Número de comentarios en la plataforma.
<code>imdb.rating</code>	float	Puntuación IMDb (0 - 10). Ej.: 9.0
<code>imdb.votes</code>	integer	Número de votos en IMDb.
<code>imdb.id</code>	integer	ID de IMDb.
<code>awards.wins</code>	integer	Premios ganados.
<code>awards.nominations</code>	integer	Nominaciones recibidas.
<code>tomatoes.viewer.rating</code>	float	Puntuación de audiencia en Rotten Tomatoes.
<code>tomatoes.critic.rating</code>	float	Puntuación de críticos en Rotten Tomatoes.

Cuadro 9: Esquema de la colección `movies`. La colección relacionada `comments` referencia `movies._id` mediante `movie_id` (ObjectId) y aporta los campos `name`, `email`, `movie_id`, `text` y `date`.

Pregunta	Query
¿Cuál es la sinopsis de Inception?	<code>{"title": "Inception"}</code>
Muéstrame las películas del género Terror estrenadas después de 2010	<code>{"genres": "Horror", "year": {"\$gt": 2010}}</code>
Películas con una puntuación IMDb superior a 8.5, ordenadas de mayor a menor	<code>[{"\$match": {"imdb.rating": {"\$gt": 8.5}}}, {"\$sort": {"imdb.rating": -1}}, {"\$project": {"title": 1, "imdb.rating": 1, "year": 1, "_id": 0}}]</code>
Las 10 películas más largas de la historia	<code>[{"\$match": {"runtime": {"\$exists": true, "\$type": "int"}}}, {"\$sort": {"runtime": -1}}, {"\$limit": 10}, {"\$project": {"title": 1, "runtime": 1, "year": 1, "_id": 0}}]</code>
¿Cuántas películas hay de cada género?	<code>[{"\$unwind": "\$genres"}, {"\$group": {"_id": "\$genres", "total": {"\$sum": 1}}}, {"\$sort": {"total": -1}}]</code>
Películas dirigidas por Christopher Nolan o Steven Spielberg	<code>{"directors": {"\$in": ["Christopher Nolan", "Steven Spielberg"]}}</code>
Películas cuyo título contiene la palabra «war» (sin distinguir mayúsculas)	<code>{"title": {"\$regex": "war", "\$options": "i"}}</code>

Cuadro 10: Ejemplos verificados de pregunta y consulta incluidos en la plantilla *few-shot*.

Campo	Tipo	Valor de ejemplo
<code>date</code>	date	2016-06-25 14:49:38
<code>email</code>	string	victoria_sanders@fakegmail.com
<code>movie_id</code>	objectid	573a139af29313caabcf0f07
<code>name</code>	string	Victoria Sanders
<code>text</code>	string	<i>Beatae omnis ad eum facilis deserunt asperiores aliquam. Esse eligendi magni doloribus. Quod voluptate veniam voluptatibus. Voluptatem cupiditate eius reiciendis nesciunt recusandae.</i>

Cuadro 11: Esquema inferido de la colección `comments` a partir de 100 documentos muestreados.

#	Pregunta	MQL generado	Desenlace
1	Las 10 películas con mayor puntuación IMDb	<code>[{"\$match": {"imdb.rating": {"\$type": "number"}}}, {"\$sort": {"imdb.rating": -1}}, {"\$limit": 10}, {"\$project": {"title": 1, "imdb.rating": 1, "year": 1, "_id": 0}}]</code>	10 documentos; la guarda numérica evita las cadenas vacías.
2a	¿Cuáles son las mejores 5 películas de Tarantino y qué rating tienen?	<code>[{"\$match": {"title": {"\$regex": "Tarantino", "\$options": "i"}}}, {"\$sort": {"imdb.rating": -1}}, {"\$limit": 5}, {"\$project": {"title": 1, "imdb.rating": 1}}]</code>	0 resultados; se comunica como mensaje informativo, no como error.
2b	¿y las de Nolan?	<code>[{"\$match": {"directors": "Christopher Nolan"}}, {"\$sort": {"awards.wins": -1}}, {"\$limit": 5}, {"\$project": {"title": 1, "awards.wins": 1}}]</code>	5 documentos; la memoria conserva movies y resuelve la anáfora hacia directors .
3	borra todas las películas	<code>[{"\$merge": {"into": "movies"}}]</code> (operador prohibido, a modo ilustrativo)	Rechazado por la capa de seguridad; ninguna escritura llega a la base de datos.

Cuadro 12: Trazas de interacción. La traza 2 es una conversación de dos turnos (2a y 2b) que ejercita la memoria conversacional; la traza 3 muestra el fallo controlado ante un operador destructivo.

Modelo	Corr. func. (%)	Coinc. exacta (%)	Corr. parcial (%)	Tasa error (%)	Latencia (s)
llama3.2 (3B)	73,7	15,8	79,0	10,5	1,27
llama3.1 (8B)	68,4	47,4	78,7	5,3	4,86
qwen2.5 (3B)	63,2	15,8	63,8	10,5	3,41
gemma2 (2B)	52,6	15,8	63,7	26,3	5,30
nl2mongo (220M)	5,3	5,3	5,7	5,3	6,26

Cuadro 13: Comparativa de modelos sobre el conjunto de prueba del *benchmark*. El modelo principal, **llama3.2**, aparece resaltado y encabeza la corrección funcional con la menor latencia. En negrita, el mejor valor de cada magnitud destacada.

 Asistente de Inteligencia NoSQL

DATASET **Películas · sample_mflix** Nueva conversación

Mejores 5 películas de christopher nolan

MQL GENERADO — COLECCIÓN: MOVIES · BD: SAMPLE_MFLIX Copiar

```
[
  {"$match": {"directors": "Christopher Nolan"}},
  {"$sort": {"imdb.rating": -1}},
  {"$limit": 5},
  {"$project": {"title": 1, "imdb.rating": 1, "year": 1, "_id": 0}}
]
```

RESULTADOS (5)

imdb.rating	year	title
9	2008	The Dark Knight
8.8	2010	Inception
8.7	2014	Interstellar
8.5	2012	The Dark Knight Rises
8.5	2006	The Prestige

y de Spielberg?

MQL GENERADO — COLECCIÓN: MOVIES · BD: SAMPLE_MFLIX Copiar

```
[
  {"$match": {"directors": "Steven Spielberg"}},
  {"$sort": {"imdb.rating": -1}},
  {"$limit": 5},
  {"$project": {"title": 1, "imdb.rating": 1, "year": 1, "_id": 0}}
]
```

RESULTADOS (5)

Consultar

Figura 21: Interfaz web de chat de MongoMind. La pregunta en lenguaje natural, el MQL generado y los resultados se presentan juntos.

